

TEMA 7

DISEÑO JERÁRQUICO Y PARAMETRIZABLE EN VHDL



UNIVERSIDAD
DE CÓRDOBA

ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba



DEPARTAMENTO DE
INGENIERÍA ELECTRÓNICA
Y DE COMPUTADORES
UNIVERSIDAD DE CÓRDOBA



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL

❖ Objetivos

- Comprender el formato descriptivo de los diseños complejos como sistemas jerárquicos
- Exponer las características del estilo estructural
- Comprender la sintaxis, funcionamiento y utilización de los componentes en VHDL
- Comprender la sintaxis y utilización de los packages en la descripción de los sistemas jerárquicos
- Exponer las características de VHDL para describir diseños escalables y parametrizables.
- Comprender la sintaxis, funcionamiento y utilización de los statements Generates.

Tema 7. Diseño jerárquico y parametrizable en VHDL

❖ Bibliografía

- John F. Wakerly. Diseño Digital. Principios y Prácticas. Ed. Prentice Hall. 2001.
- William Kafig. VHDL 101. Everything you need to know to get started. Ed. Newnes. 2011.
- Gina R. Smith. FPGA 101. Everything you need to know to get started. Ed. Newnes. 2010.
- Pong P. Chu. FPGA Prototyping by VHDL Examples. Ed. Wiley. 2008.
- Ion Grout. Digital Systems Design with FPGAs and CPLDs. Ed Elsevier Ltd. 2008.
- Peter Wilson. Design Recipes for FPGAs. Ed. Newnes. 2007.
- David Pellerin y otros. VHDL Made Easy!. Ed. Mc Graw Hill. 1997.
- P. J. Ashenden. The Student's Guide to VHDL. Ed. Morgan Kaufmann. 2008.
- Peter J. Ashenden. The Designer's Guide to VHDL. Ed. Elsevier Inc. 2008.
- Douglas Perry. VHDL. Ed. Mc-Graw Hill. 1998.
- R. C. Cofer and Benjamin F. Harding. Rapid System Prototyping with FPGAs. Ed. Elsevier Inc. 2006.
- Pong P. Chu. RTL Hardware Design Using VHDL. Ed. Wiley. 2006.
- Sundar Rajan. Essential VHDL. Rtl Syntesis Done Right. 1998
- F. Pardo y J.A. Boluda. VHDL para la Síntesis y Modelado de Circuitos. Ed. Ra-ma. 1999

1. Introducción

❖ Los sistemas electrónicos digitales complejos es conveniente diseñarlos como **diseños jerárquicos**:

- ✓ El diseño se **divide en varios niveles**, cada uno de los cuales utiliza unos **bloques o componentes específicos descritos en un nivel inferior**.
- ✓ Hay un **nivel superior (raíz)** y **uno o más niveles inferiores**.
- ✓ Cada **nivel sucesivo** representa bloques o componentes **más simples** que los del **nivel anterior**.
- ✓ El **último nivel (inferior)** describe los componentes **más simples**.

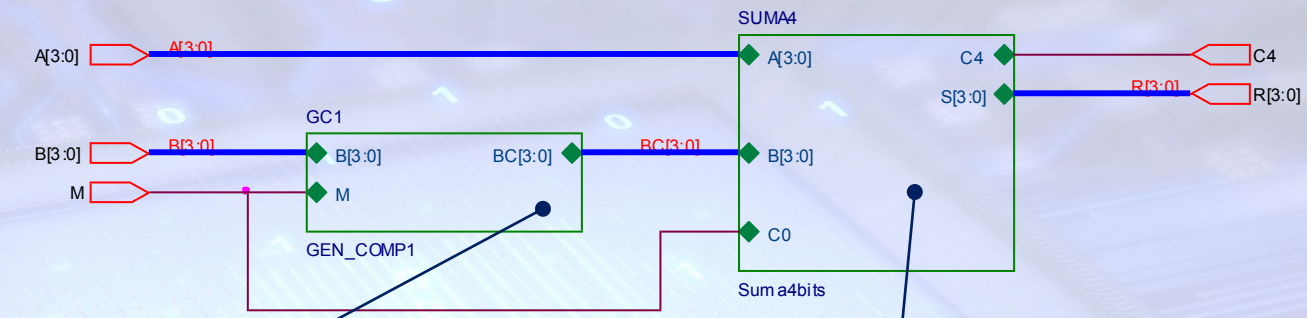
■ **Ventajas del diseño jerárquico:**

- ✓ Diseño simultáneo de cada bloque o componente.
- ✓ Verificación individual de cada componente.
- ✓ Construcción del diseño por fases.
- ✓ Se facilita el diseño y el trabajo en equipo.
- ✓ Reutilización de los componentes.

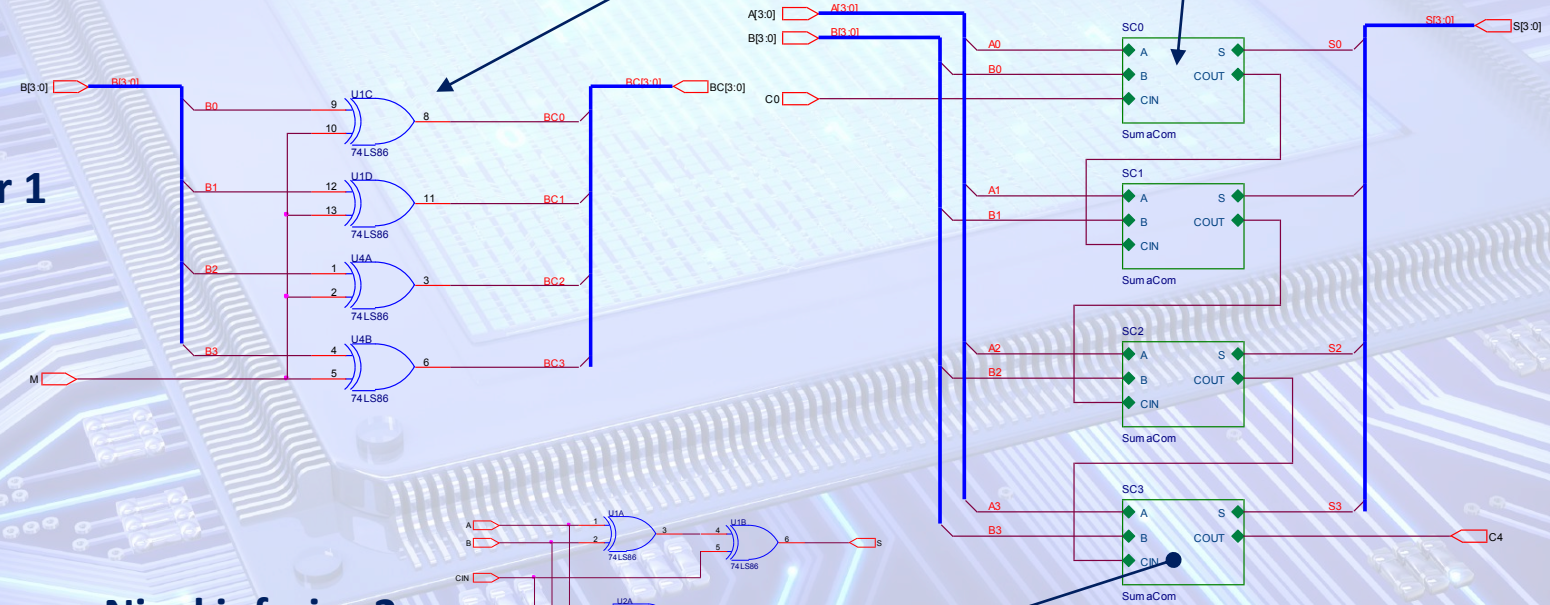
1. Introducción

❖ Ejemplo de diseño jerárquico. Sumador-restador mediante el C2

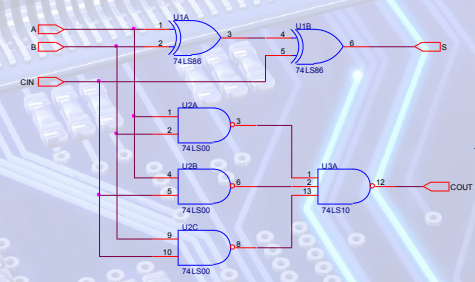
■ Nivel raíz



■ Nivel inferior 1



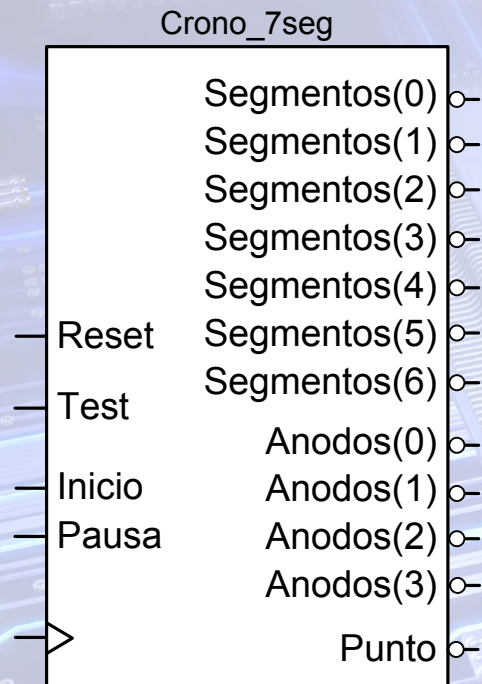
■ Nivel inferior 2



1. Introducción

❖ Ejemplo de diseño jerárquico. Cronómetro con cuenta de minutos y segundos y salidas para un visualizador de 7 segmentos de 4 dígitos.

▪ La representación a nivel de bloque corresponde a la entidad del nivel superior (raíz)



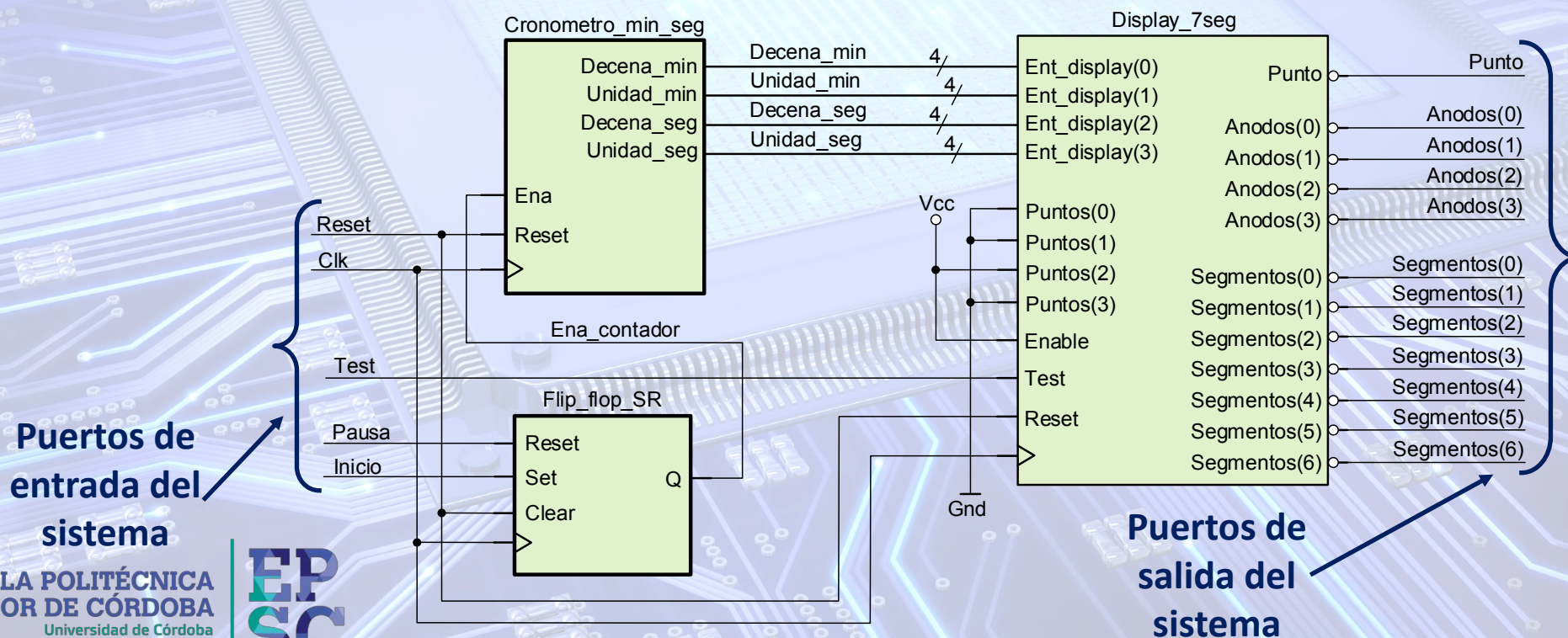
- ✓ **CLK:** Es la señal de reloj de todo el sistema. Su frecuencia es de 50 MHz.
- ✓ **Reset:** Pone el cronómetro a cero e inicializa el sistema.
- ✓ **Test:** Permite comprobar el estado de los segmentos del display. Si se activa se encienden todos los segmentos.
- ✓ **Inicio:** Si se activa el cronómetro empieza a contar el tiempo. Si se desactiva el cronómetro sigue funcionando salvo que se pulse Pausa.
- ✓ **Pausa:** Si se activa se detiene la cuenta. Una vez deshabilitada la cuenta, si se deja de pulsar pausa el cronómetro sigue deshabilitado.
- ✓ **Segmentos(6..0):** Terminales de control de los segmentos del display.
- ✓ **Anodos(3..0):** Terminales de control de los ánodos del display.
- ✓ **Punto:** Terminal de control de los puntos del display.

1. Introducción

❖ Ejemplo de diseño jerárquico. Cronómetro con cuenta de minutos y segundos y salidas para un visualizador de 7 segmentos de 4 dígitos.

▪ Diagrama de bloques del sistema.

- ✓ Corresponde a la descripción del nivel raíz.
- ✓ Se realiza mediante el estilo estructural conectando los tres bloques o componentes a las entradas y salidas del sistema, y las salidas de unos componentes con las entradas de los otros.

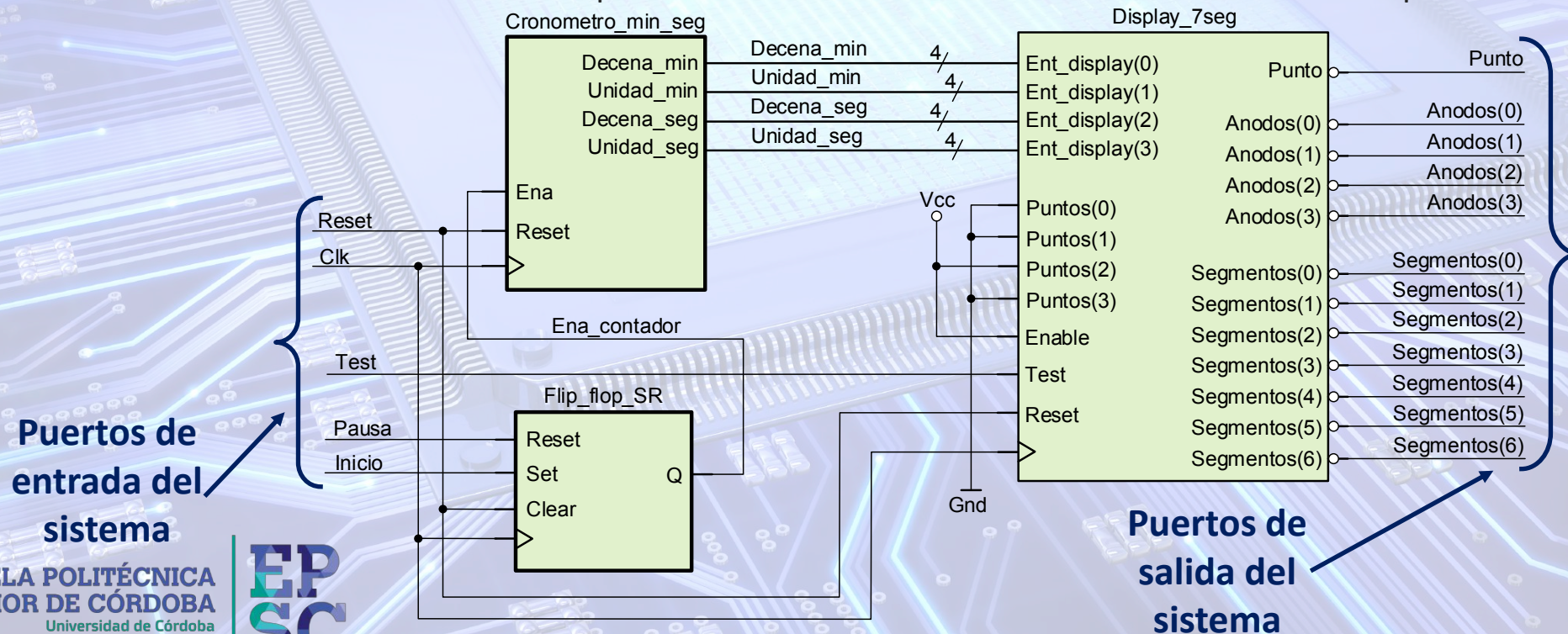


1. Introducción

❖ Ejemplo de diseño jerárquico. Cronómetro con cuenta de minutos y segundos y salidas para un visualizador de 7 segmentos de 4 dígitos.

- En el nivel inferior siguiente al raíz se describe cada uno de los tres componentes.

- ✓ Se puede subdividir en otros bloques o describir directamente su comportamiento en VHDL, dependiendo de la complejidad.
- ✓ En el nivel más inferior se describe en VHDL el comportamiento de los componentes básicos usados en el nivel inmediatamente superior.

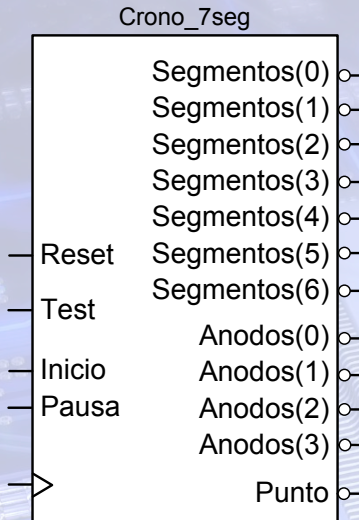


1. Introducción

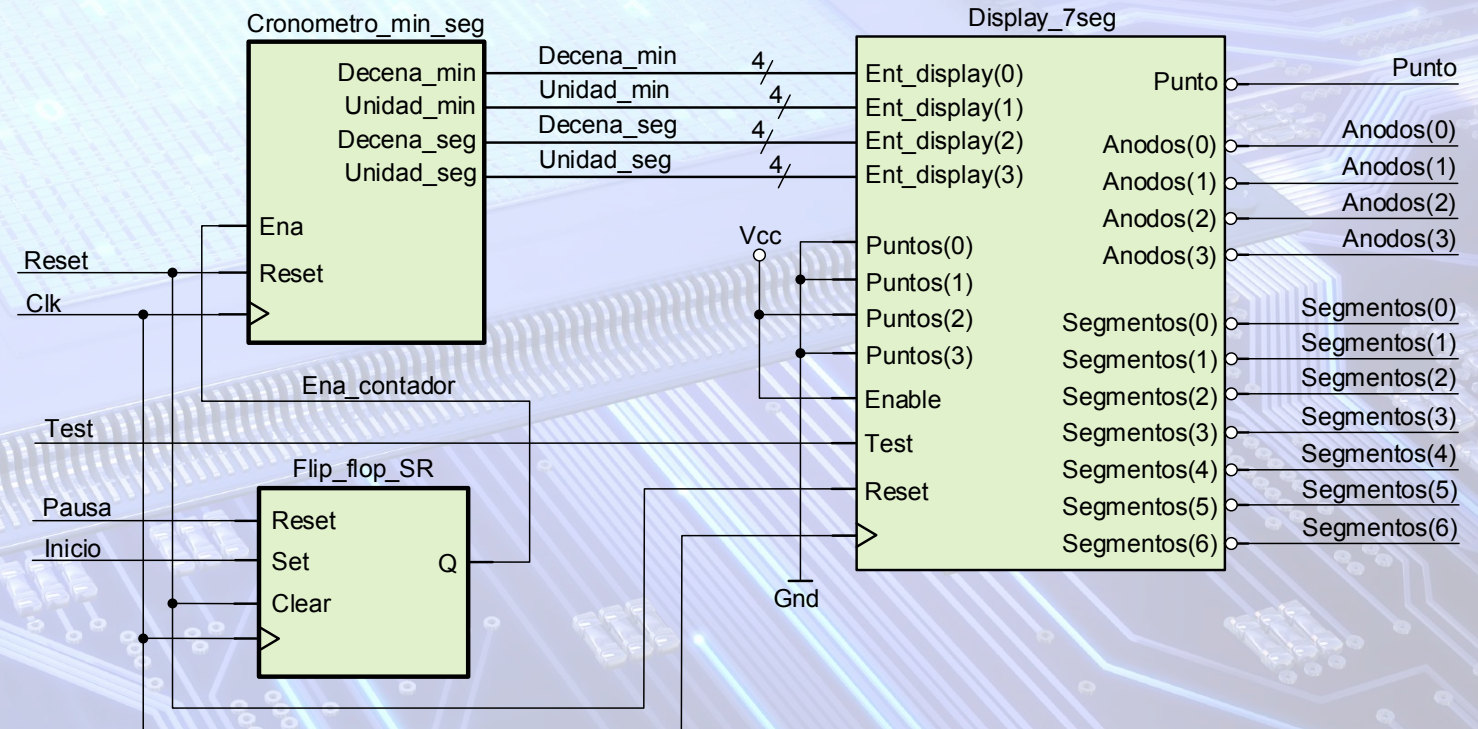
❖ Ejemplo de diseño jerárquico. Cronómetro con cuenta de minutos y segundos y salidas para un visualizador de 7 segmentos de 4 dígitos.

▪ La jerarquía del diseño también se puede mostrar en forma de árbol.

- ✓ En nuestro ejemplo, la jerarquía constará de dos niveles
- ✓ Pero los sistemas muy complejos pueden tener más de dos niveles, y no tienen por qué ser homogéneas todas las ramas.
- ✓ Es decir, puede haber unas ramas con más niveles que otras.



Jerarquía del cronómetro en forma de árbol



2. Jerarquía en VHDL

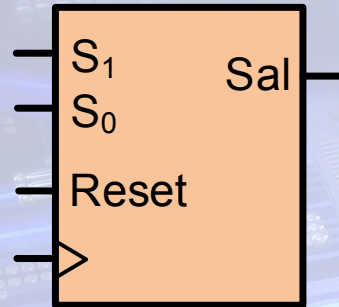
- ❖ En VHDL los sistemas jerárquicos se describen mediante el **estilo estructural**
 - Se indican los **componentes** de cada nivel y **cómo se interconectan** con los de su mismo nivel, y **cómo se realizan** con los componentes del nivel inferior.
- ❖ Las entidades de diseño que se emplean en VHDL para construir sistemas jerárquicos son los **componentes**
 - Un componente consta de un par entidad/arquitectura
 - Proceso para emplear un componente:
 - ✓ Hacerlo **visible** mediante un **statement de declaración de componentes**:
 - En el **área declarativa** de la arquitectura, o
 - en un **package externo**
 - ✓ **Crear una copia o instancia del componente** mediante un statement de instanciación de componentes.

2. Jerarquía en VHDL

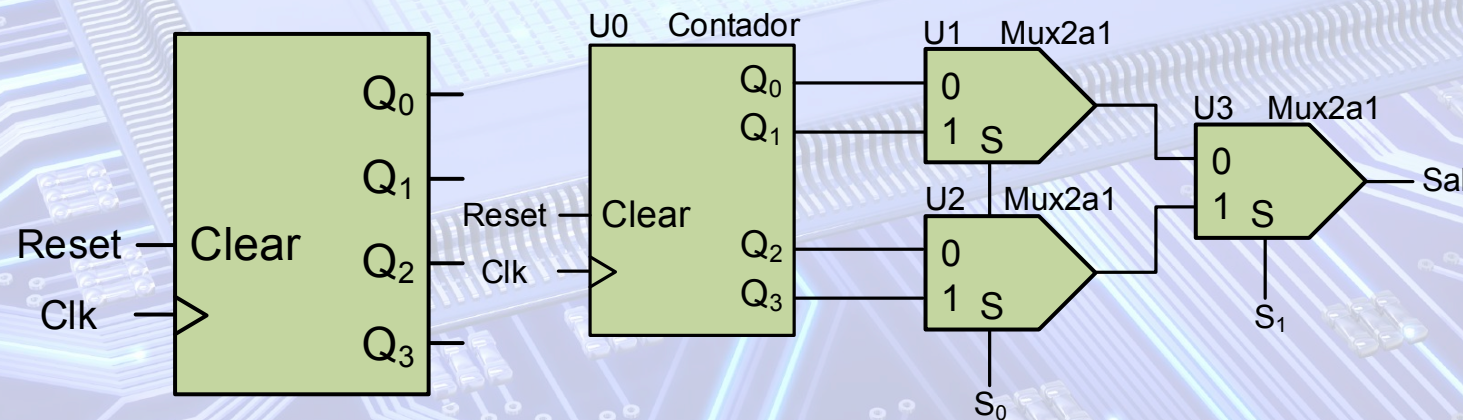
❖ Ejemplo. Divisor de frecuencia

- Genera en su salida una señal de reloj cuya frecuencia está dividida respecto a la señal de reloj CLK por una potencia de 2 en función del valor de las entradas de control de selección, S_1 y S_0 .
- Aunque lo lógico sería describirlo mediante un único par entidad-arquitectura, se va describir como un diseño jerárquico para mostrar la metodología.
- Se comienza determinando la estructura del nivel superior usando los componentes del nivel inferior:

Divisor frecuencia



S_1	S_0	f_{SAL}
0	0	$f_{clk}/2$
0	1	$f_{clk}/4$
1	0	$f_{clk}/8$
1	1	$f_{clk}/16$



2. Jerarquía en VHDL

❖ Ejemplo. Divisor de frecuencia

- Por tanto, consta de dos niveles:

- ✓ El nivel superior (raíz):

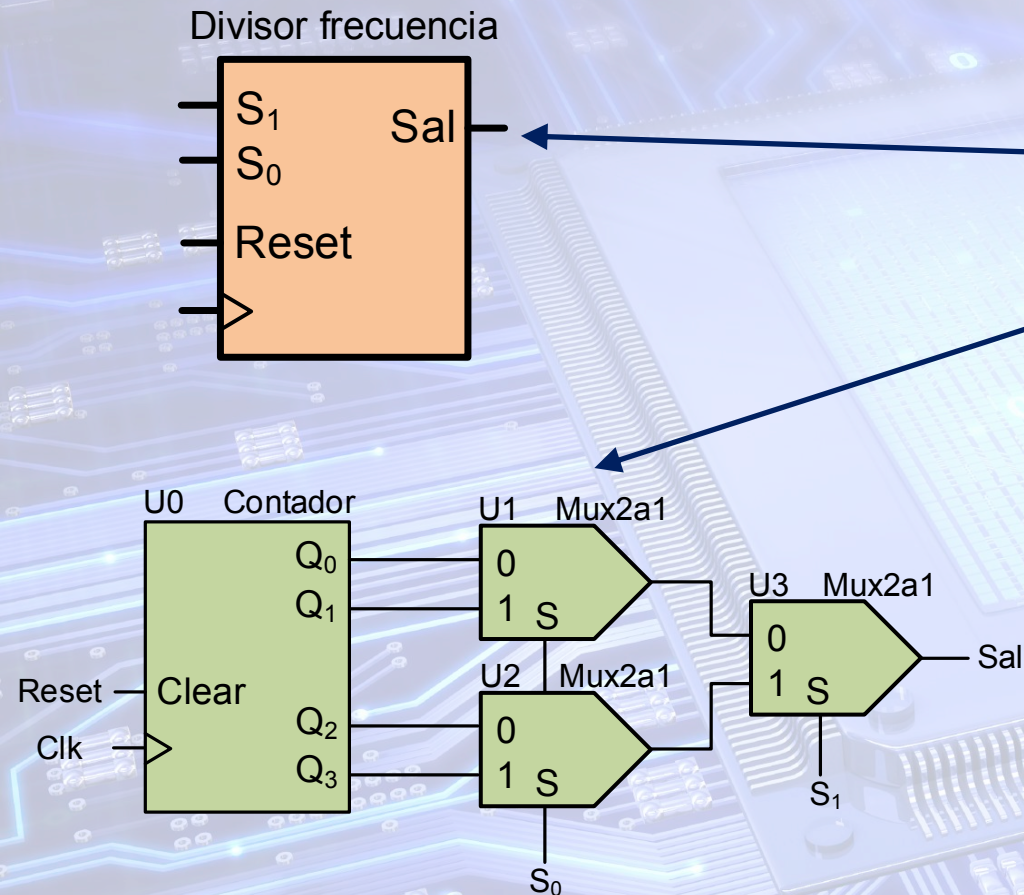
- La entidad describe las conexiones externas del sistema.
- La arquitectura describe la estructura del sistema mostrada en la figura anterior.

- ✓ El nivel inferior describe cada uno de los componentes básicos:

- Contador
- Mux2a1

- En el nivel superior se usa 3 componentes Mux2a1

- ✓ Se deben crear tres instancias del componente.



2. Jerarquía en VHDL

❖ Ejemplo. Divisor de frecuencia

- Hay varias formas de realizar la descripción:

- ✓ Un único fichero
- ✓ Múltiples ficheros 👉 Es la más usada
- ✓ En este ejemplo, se usará un único fichero.
- ✓ Posteriormente, se usará la otra alternativa.

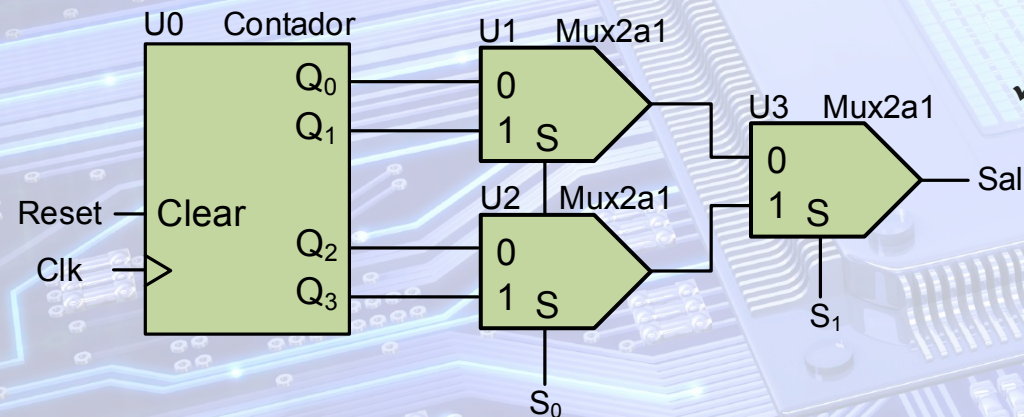
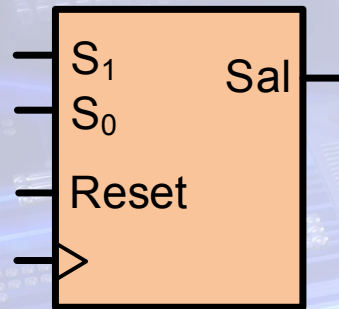
- Metodología descriptiva usando un solo fichero:

- ✓ Se comienza describiendo cada componente básico del nivel inferior mediante su par entidad-arquitectura.
- ✓ A continuación se describe la entidad del sistema y su arquitectura.

- La arquitectura describe la **estructura del sistema** en estilo estructural.

- Primero se declara los dos componentes básicos (Contador y Mux2a1) en el área de declaración.
- Se instancian los componentes y se conectan.

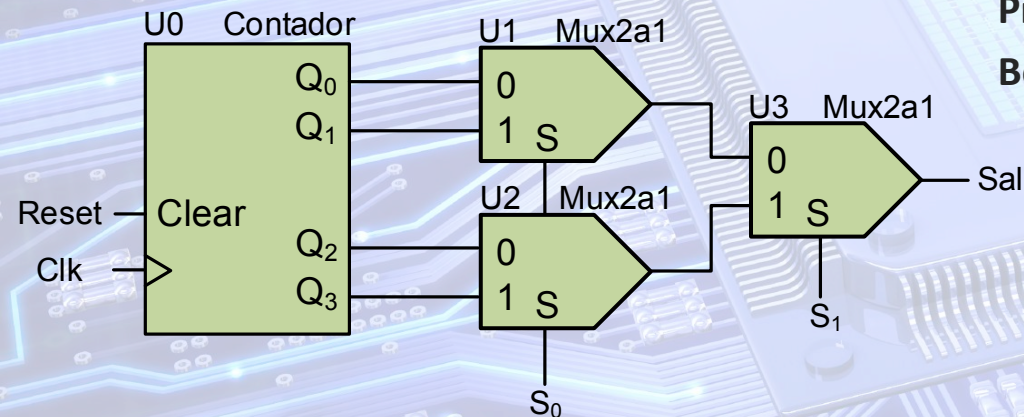
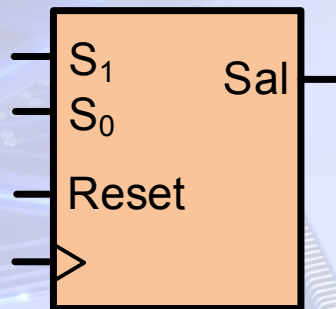
Divisor frecuencia



2. Jerarquía en VHDL. Ejemplo. Divisor de frecuencia

 **Fichero fuente** →
Divisorcompleto.vhd

Divisor frecuencia



```
Library ieee;
```

```
Use ieee.std_logic_1164.all;
```

```
Use ieee.std_logic_unsigned.all;
```

```
Entity Contador is port (
```

```
  Clk, Clear : in std_logic;
```

```
  Q : out std_logic_vector(3 downto 0) );
```

```
End Contador;
```

```
Architecture Funcional of Contador is
```

```
  Signal Cuenta : std_logic_vector(3 downto 0);
```

```
Begin
```

```
  Process (Clk)
```

```
  Begin
```

```
    If Clk'event and Clk = '1' then
```

```
      If Clear = '1' then
```

```
        Cuenta <= (others => '0');
```

```
      Else
```

```
        Cuenta <= Cuenta + 1;
```

```
      End if;
```

```
    End if;
```

```
  End process;
```

```
  Q <= Cuenta;
```

```
End Funcional;
```

2. Jerarquía en VHDL. Ejemplo. Divisor de frecuencia

💡 Fichero fuente

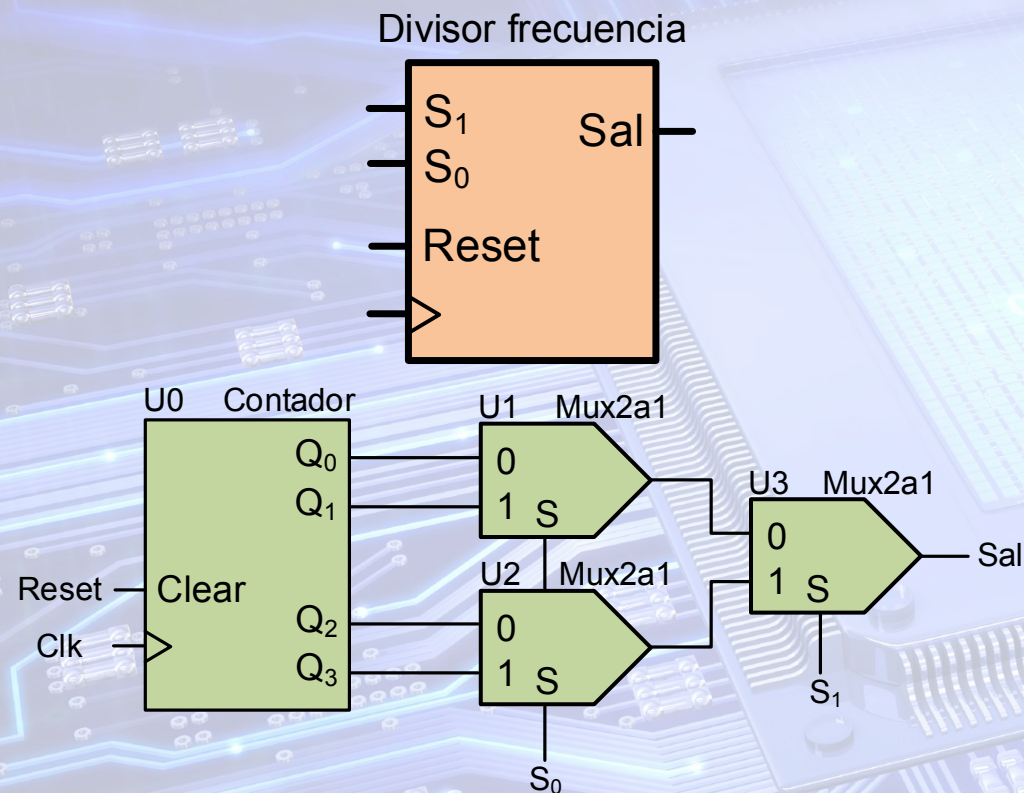
Divisorcompleto.vhd(continuación)

```

Library ieee;
Use ieee.std_logic_1164.all;

Entity Mux2a1 is port (
    S : in std_logic;
    E : in std_logic_vector(1 downto 0);
    Y : out std_logic );
End Mux2a1;

Architecture Funcional of Mux2a1 is
Begin
    Y <= E(0) when S = '0'
        else E(1);
End Funcional;
  
```

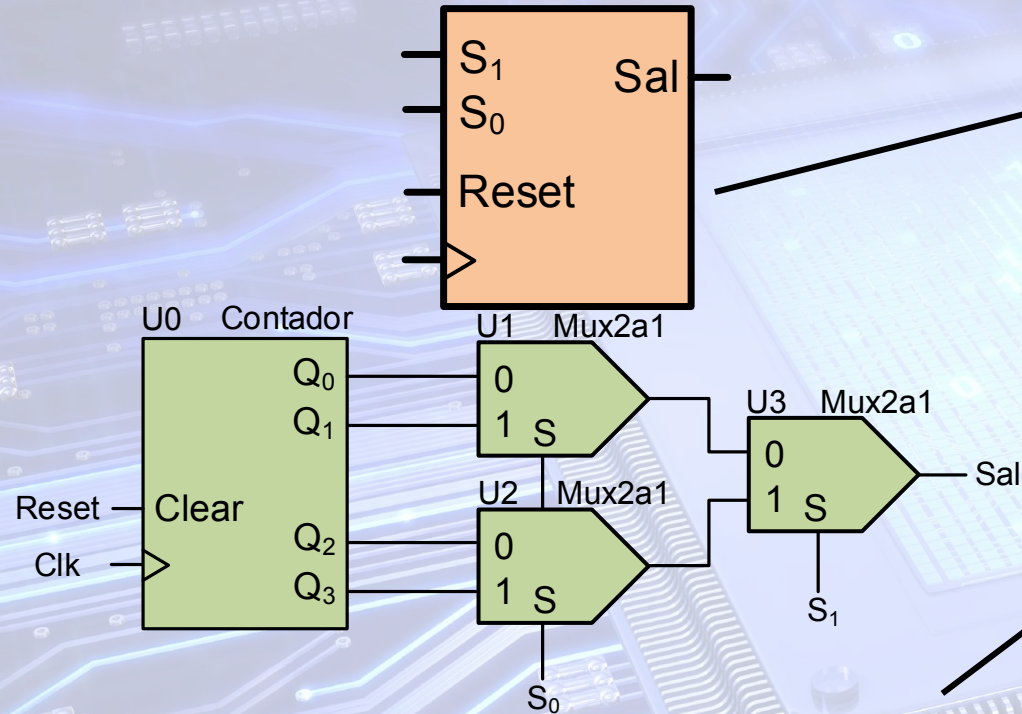


2. Jerarquía en VHDL. Ejemplo. Divisor de frecuencia

Fichero fuente

Divisorcompleto.vhd(continuación)

Divisor frecuencia



Se debe declarar las señales Q y Mux para conectar los componentes entre sí.

Estas señales son equivalentes a los hilos de un esquema eléctrico.

-- Nivel raíz

Library ieee;

Use ieee.std_logic_1164.all;

Entity Divisor **is port** (

Reset, Clk : **in** std_logic;

S : **in** std_logic_vector(1 **downto** 0);

Sal : **out** std_logic);

End Divisor;

Architecture Estructural **of** Divisor **is**

Signal Q : std_logic_vector(3 **downto** 0);

Signal Mux : std_logic_vector(1 **downto** 0);

-- Se declaran los componentes

Component Contador **port** (

Clk, Clear : **in** std_logic;

Q : **out** std_logic_vector(3 **downto** 0));

End Component;

Component Mux2a1 **port** (

S : **in** std_logic;

E : **in** std_logic_vector(1 **downto** 0);

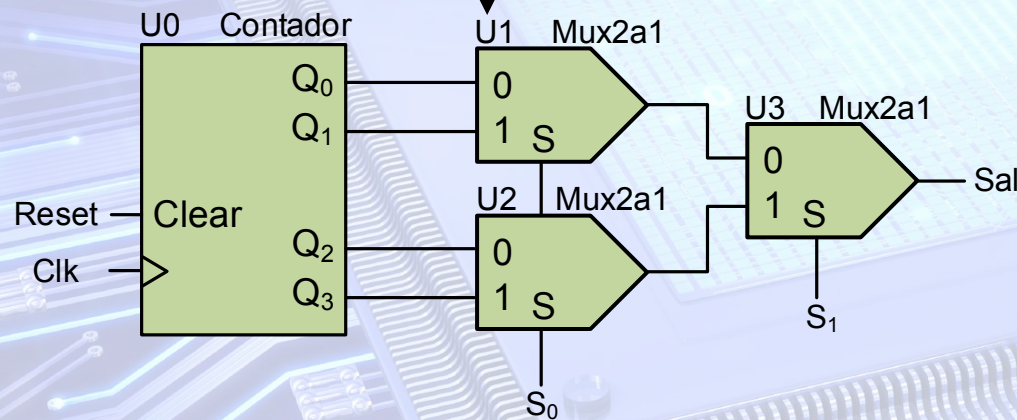
Y : **out** std_logic);

End Component;



Fichero fuente Divisorcompleto.vhd (continuación)

La etiqueta es equivalente a la referencia del componente en el esquema eléctrico.



La etiqueta permite instanciar varias veces un mismo componente. En el ejemplo, 3 Mux2a1.

Begin

-- se instancia el contador U0

U0 : Contador **port map** (

Clk => Clk,

Clear => Reset,

Q => Q);

-- se instancia el mux U1

U1 : Mux2a1 **port map** (

E => Q(1 **downto** 0),

S => S(0),

Y => Mux(0));

-- se instancia el mux U2

U2 : Mux2a1 **port map** (

E => Q(3 **downto** 2),

S => S(0),

Y => Mux(1));

-- se instancia el mux U3

U3 : Mux2a1 **port map** (

E => Mux,

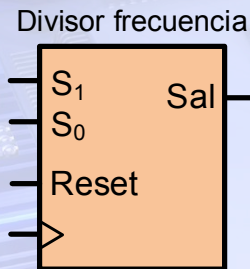
S => S(1),

Y => Sal);

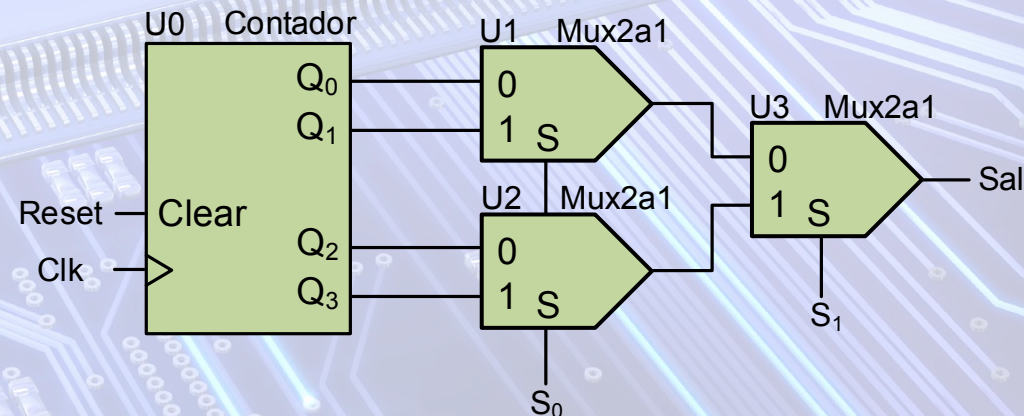
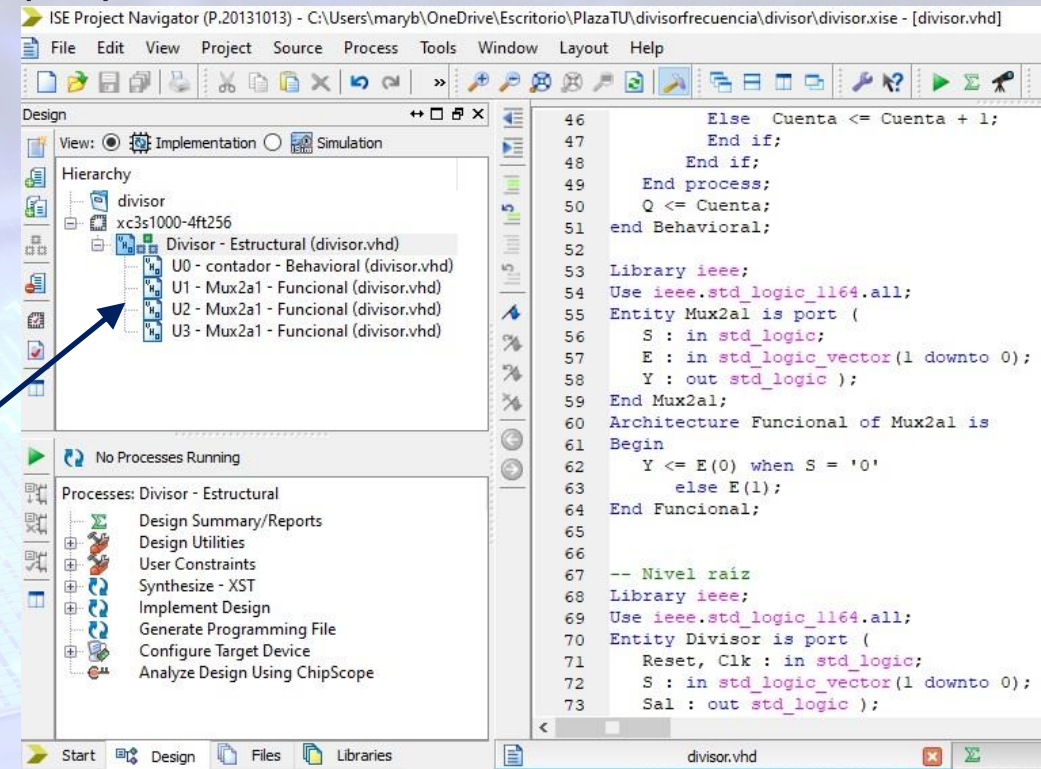
End Estructural;

2. Jerarquía en VHDL. Ejemplo. Divisor de frecuencia

- ❖ Ventana del navegador de proyectos de ISE usando sólo el fichero Divisorcompleto.vhd



Muestra la jerarquía del
diseño en forma de árbol



3. Statement de declaración de componentes

- Una declaración de componente **especifica el componente**, que se va usar en ese nivel, indicando **su interfaz externo** mediante constantes genéricas y ports.
- Se emplea en el **área declarativa de la arquitectura** o en un **package**.
- **Sintaxis**

```
component identificador [ is ]  
    [ generic (lista_interface_generic) ; ]  
    port (lista_interface_port ) ;  
end component [identificador] ;
```

- **Identificador** es el nombre del componente que debe coincidir con el de su entidad.
- La **lista_interface_generic** permite definir componentes genéricos, como por ejemplo, un contador binario de n bits.
- La **lista_interface_port** especifica las conexiones del componente, que debe coincidir con las de su entidad.
- **VHDL-87**
 - No se puede usar **is** ni el nombre del componente al final de la declaración.

3. Statement de declaración de componentes

❖ Ejemplos

```
Entity Contador port (  
    Clk, Clear : in std_logic;  
    Q : out std_logic_vector(3 downto 0) );  
End;  
  
Entity Mux2a1 port (  
    S : in std_logic;  
    E : in std_logic_vector(1 downto 0);  
    Y : out std_logic );  
End;  
  
Entity Contador  
    generic ( N : integer := 4);  
    port (Clk, Clear : in std_logic;  
        Q : out std_logic_vector(N - 1 downto 0));  
End;
```

```
Component Contador port (  
    Clk, Clear : in std_logic;  
    Q : out std_logic_vector(3 downto 0) );  
End Component;  
  
Component Mux2a1 port (  
    S : in std_logic;  
    E : in std_logic_vector(1 downto 0);  
    Y : out std_logic );  
End Component;  
  
Component Contador  
    generic ( N : integer := 4);  
    port (Clk, Clear : in std_logic;  
        Q : out std_logic_vector(N - 1 downto 0)  
        );  
End Component;
```

4. Statement de instanciación de componentes

- Referencia componentes del nivel inferior de diseño y crea una copia o instancia de dichos componentes.
- **Sintaxis**

```
Etiqueta_instancia: [component] nombre_componente  
                    [ generic map (lista_asociación_parámetros)]  
                    port map (lista_asociación_port) ;
```

- ✓ **Etiqueta_instancia** es el nombre que se le asigna a la copia del componente, y es su referencia.
- ✓ Para asociar el componente con su entidad debe coincidir el nombre del componente con el de su entidad.
- **Port map**
 - ✓ Indica las conexiones del componente.
 - ✓ Las conexiones se indican entre paréntesis a continuación de las palabras claves port map.
 - ✓ Hay dos formas de asignar los port a un componente:
 - Asociación posicional.
 - Asociación por nombres.

4. Statement de instanciación de componentes

❖ Port map

▪ Asociación posicional

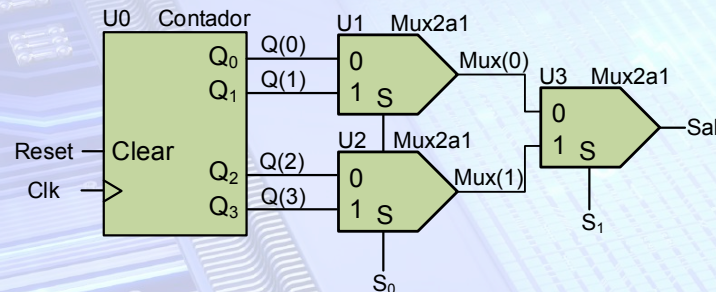
- ✓ Asigna los **ports de la entidad y/o señales de la architecture** de ese nivel (**parámetros actuales**) a los correspondientes **ports del componente** del nivel inferior (**parámetros formales**) por su **posición en la lista de port de la declaración de su entidad**.
- ✓ Se debe indicar el **mismo número y tipos de port** que se declararon en la entidad del componente del nivel inferior.
- ✓ **Ventajas**
 - Es muy fácil y rápida de usar.
- ✓ **Desventajas**
 - No se realiza comprobación de errores.
 - Si inadvertidamente se invierten el orden de dos ports, puede que no se produzca ningún error de compilación, pero el diseño será incorrecto.

4. Statement de instanciación de componentes

❖ Ejemplos de asignación posicional

-- Declaración de la entidad del contador

```
Entity Contador port (
  Clk, Clear : in std_logic;
  Q : out std_logic_vector(3 downto 0) );
End;
```



-- Declaración de la entidad del multiplexor de 2 a 1

```
Entity Mux2a1 port (
  S : in std_logic;
  E : in std_logic_vector(1 downto 0);
  Y : out std_logic );
End;
```

```
Component Contador port (
  Clk, Clear : in std_logic;
  Q : out std_logic_vector(3 downto 0) );
End Component;
```

```
Component Mux2a1 port (
  S : in std_logic;
  E : in std_logic_vector(1 downto 0);
  Y : out std_logic );
End Component;

signal Q : std_logic_vector(3 downto 0) ;
signal Mux : std_logic_vector(1 downto 0) ;
```

-- se instancia el contador U0

```
U0 : Contador port map (
  Clk, Reset, Q );
```

-- se instancia el mux U1

```
U1 : Mux2a1 port map (
  S(0), Q(1 downto 0), Mux(0) );
```

4. Statement de instanciación de componentes

❖ Port map

■ Asociación por nombres

- ✓ Se indica tanto el nombre actual como el formal mediante el operador `=>`.

- `Puerto_formal => Puerto_señal_actual`
- `Nivel inferior => Nivel inmediatamente superior`

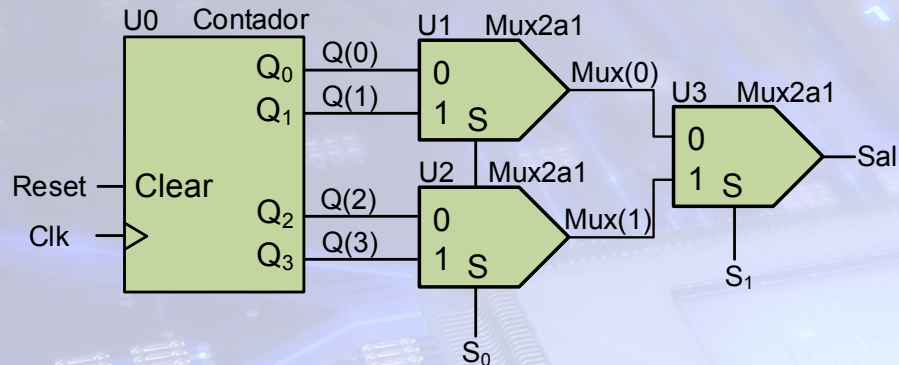
✓ Ventajas

- Detecta los errores anteriores.
- Se puede intercambiar el orden de los ports.
- Se pueden omitir los ports que no se empleen, indiferentemente de su orden.

- ✓ Es aconsejable emplear la asociación por nombres.

4. Statement de instanciación de componentes

❖ Ejemplos de asignación por nombres



Component Contador port (

Clk, Clear : **in** std_logic;

Q : **out** std_logic_vector(3 **downto** 0));

End Component;

Component Mux2a1 port (

S : **in** std_logic;

E : **in** std_logic_vector(1 **downto** 0);

Y : **out** std_logic);

End Component;

signal Q : std_logic_vector(3 **downto** 0) ;

signal Mux : std_logic_vector(1 **downto** 0) ;

-- se instancia el contador U0

U0 : Contador **port map** (

Clk => Clk,

Clear => Reset,

Q => Q);

-- se instancia el mux U1

U1 : Mux2a1 **port map** (

E => Q(1 **downto** 0),

S => S(0),

Y => Mux(0));

-- se instancia el mux U2

U2 : Mux2a1 **port map** (

E => Q(3 **downto** 2),

S => S(0),

Y => Mux(1));

-- se instancia el mux U3

U3 : Mux2a1 **port map** (

S => S(1),

E => Mux,

Y => Sal);

4. Statement de instanciación de componentes

Sintaxis

```
Etiqueta_instancia: [component] nombre_componente  
                    [ generic map lista_asociación_parámetros)]  
                    port map (lista_asociación_port) ;
```

■ Generic map

- ✓ Se emplean para pasar información (parámetros genéricos) a una instancia de un componente del nivel inferior.
- ✓ Particularizan la copia del componente.

• Aclaración

- Los statements de instanciación de componentes son concurrentes, por lo que es indiferente su orden dentro del cuerpo de la architecture.

4. Statement de instanciación de componentes

❖ Ejemplos

Component Mux2a1_nbits

generic (N : integer := 8);

port (S : **in** std_logic;

E : **in** std_logic_vector(N-1 **downto** 0);

Y : **out** std_logic_vector(N-1 **downto** 0));

End Component;

Component Contador_nbits

generic (N : integer := 4);

port (Clk, Clear : **in** std_logic;

Q : **out** std_logic_vector(N-1 **downto** 0));

End Component;

-- se instancia el mux U1 de 4 bits

U1 : Mux2a1_nbits

generic map (N => 4)

port map (

E => Entrada,

S => Seleccion,

Y => Salida);

-- se instancia el contador U2 de 8 bits

U2 : Contador_nbits

generic map (N => 8)

port map (

Clk => Clk,

Clear => Reset,

Q => Q);

4. Statement de instanciación de componentes

❖ Salidas no conectadas

- En la instancia de un componente puede ocurrir que no se emplee alguna de sus salidas por lo que no debe conectarse.
- Las **salidas no conectadas** se especifican mediante la **palabra clave open**.
- **Ejemplo**
 - ✓ No se usa la salida de acarreo de un contador.

```
Component Contador port (  
    Clk, Clear : in std_logic;  
    Q : out std_logic_vector(3 downto 0);  
    Carry : out std_logic );  
End Component;
```

```
-- se instancia el contador U0  
U0 : Contador port map (  
    Clk => Clk,  
    Clear => Reset,  
    Q => Q,  
    Carry => open );
```

4. Statement de instanciación de componentes

❖ Entradas no conectadas

- Si no se emplea alguna **entrada** de un componente **no se puede dejar sin conectar**.
- Debe conectarse a **'1'** (VCC) o **'0'** (GND).
- **VHDL-87**
 - ✓ Se define una **señal interna** o una **constante** (VCC o GND) y se inicializa al valor correspondiente.
 - ✓ Se conecta la entrada no usada a la señal o constante.

-- Ejemplo inválido

Architecture Funcional

```
Component Contador port (  
    Clk, Clear : in std_logic;  
    Q : out std_logic_vector(3 downto 0) );
```

End Component;

```
Constant Vcc : std_logic := '1';
```

Begin

```
U0 : Contador port map (  
    Clk => Clk, Clear => open, Q => Q);
```

End;

-- Ejemplo válido

Architecture Funcional

```
Component Contador port (  
    Clk, Clear : in std_logic;  
    Q : out std_logic_vector(3 downto 0) );
```

End Component;

```
Constant Gnd : std_logic := '0';
```

Begin

```
U0 : Contador port map (  
    Clk => Clk, Clear => Gnd, Q => Q);
```

End;

4. Statement de instanciación de componentes

❖ Entradas no conectadas

▪ VHDL-93

- ✓ Las entradas no conectadas se pueden conectar directamente a '0' o '1'.

-- Ejemplo válido VHDL-93

Architecture Funcional

Component Contador **port** (

Clk, Clear : **in** std_logic;

Q : **out** std_logic_vector(3 **downto** 0));

End Component;

Begin

U0 : Contador **port map** (

Clk => Clk, Clear => '0', Q => Q);

End;

5. Descripción de sistemas jerárquicos mediante múltiples ficheros

❖ Características

- Es el método más usado para describir los sistemas jerárquicos en VHDL.
- Cada bloque o componente de un nivel se describe en un fichero separado.

❖ Ventajas frente a un único fichero

- Se reduce el tamaño de los ficheros.
- Descripción de los bloques o componentes más legible.
- Usando **Packages** permite **reutilizar** más fácilmente los bloques o componentes en otros diseños.

❖ Procedimiento

- Antes de usarse los componentes en una arquitectura del nivel superior tienen que **analizarse por la herramienta de síntesis y almacenarse en la librería work**.
- Para usar el componente se debe usar la cláusula use:
 - ✓ Use work.nombre_componente;

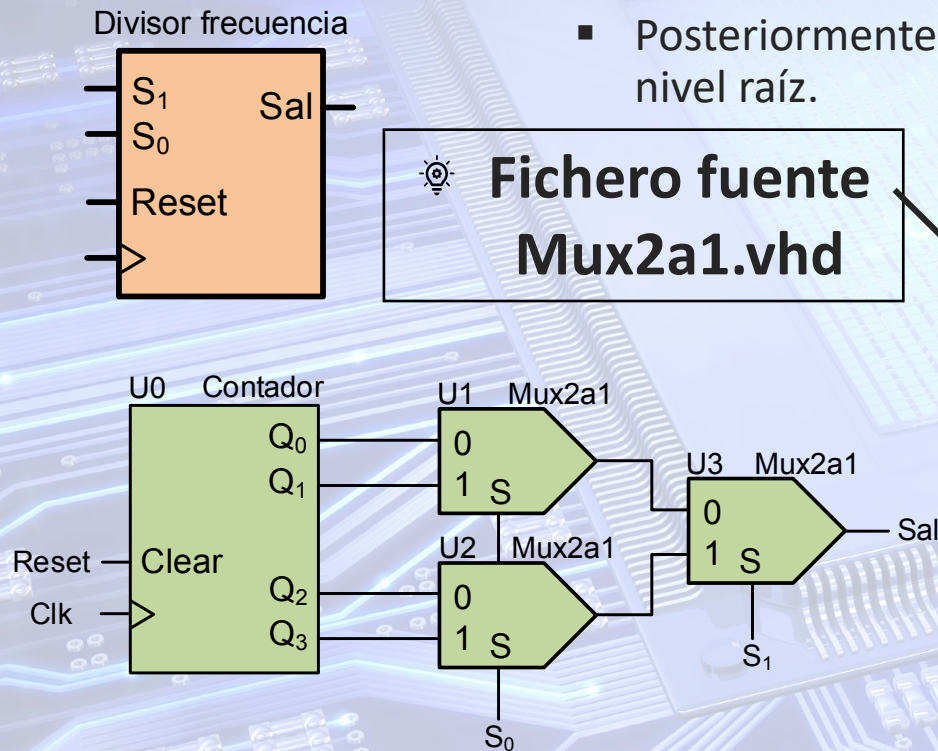
5. Descripción de sistemas jerárquicos mediante múltiples ficheros

❖ Metodología de descripción con ISE

- ISE de Xilinx usa **XST** como **herramienta de síntesis**.
 - **XST**, como la gran mayoría de las herramientas de síntesis, **automáticamente**:
 - ✓ **Determina si se describe un sistema jerárquico.**
 - ✓ **Detecta los niveles de jerarquía.**
 - ✓ **Analiza los pares entidad-arquitectura de cada componente,** descritos en ficheros separados, antes de analizar los niveles jerárquicos superiores.
 - ✓ **Almacena el resultado en la librería work.**
- 💡 Como la **librería work** es visible por defecto **no es necesario usar la cláusula use.**

5. Descripción de sistemas jerárquicos mediante múltiples ficheros. Descripción del divisor de frecuencia

- ❖ Se describe en VHDL cada uno de los bloques o componentes.
 - Conviene usar el mismo nombre para la entidad y el fichero.
- ❖ Se hará una descripción ascendente:
 - Se comienza describiendo los componentes del nivel inferior.
 - Posteriormente, se describe el sistema final que corresponde al nivel raíz.



```

Library ieee;
Use ieee.std_logic_1164.all;

Entity Mux2a1 is port (
    S : in std_logic;
    E : in std_logic_vector(1 downto 0);
    Y : out std_logic );
End Mux2a1;

Architecture Funcional of Mux2a1 is
Begin
    Y <= E(0) when S = '0'
        else E(1);
End Funcional;
  
```

5. Descripción de sistemas jerárquicos mediante múltiples ficheros. Divisor de frecuencia

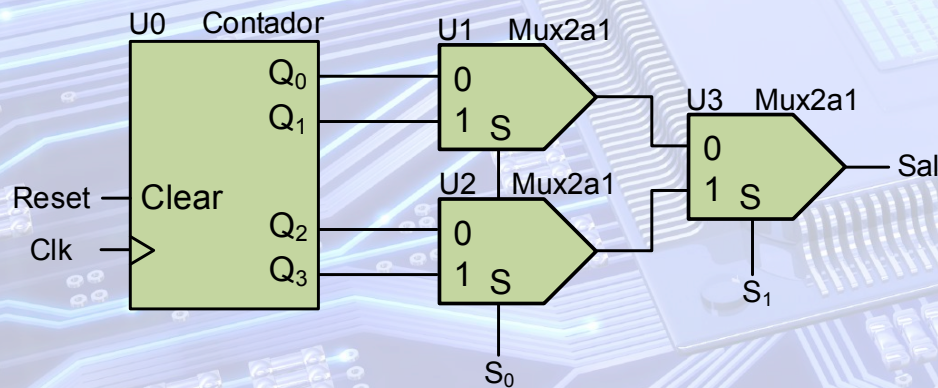
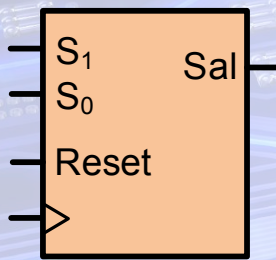


**Fichero fuente
Contador.vhd**

```

Library ieee;
Use ieee.std_logic_1164.all;
Use ieee.std_logic_unsigned.all;
Entity Contador is
  generic (N : natural := 8);
  port ( Clk, Clear : in std_logic;
        Q : out std_logic_vector(N-1 downto 0) );
End Contador;
Architecture Funcional of Contador is
  Signal Cuenta : std_logic_vector(N-1 downto 0);
Begin
  Process (Clk)
  Begin
    If Clk'event and Clk = '1' then
      If Clear = '1' then
        Cuenta <= (others => '0');
      Else
        Cuenta <= Cuenta + 1;
      End if;
    End if;
  End process;
  Q <= Cuenta;
End Funcional;
  
```

Divisor frecuencia

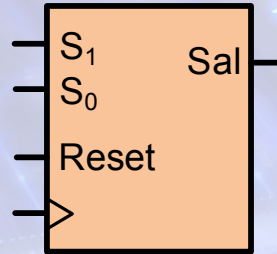


5. Descripción de sistemas jerárquicos mediante múltiples ficheros. Divisor de frecuencia



Fichero fuente
Divisor.vhd

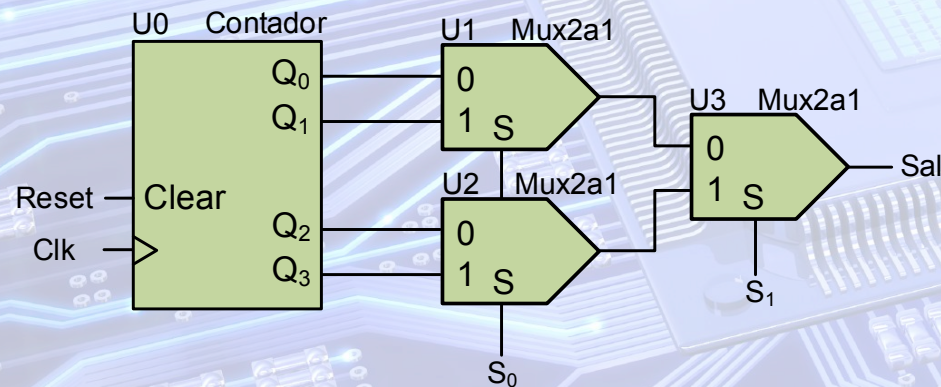
Divisor frecuencia



Se debe declarar las señales Q y Mux para conectar los componentes entre sí.



Estas señales son equivalentes a los hilos de un esquema eléctrico.



-- Nivel raíz

Library ieee;

Use ieee.std_logic_1164.all;

Entity Divisor **is port** (

Reset, Clk : **in** std_logic;

S : **in** std_logic_vector(1 **downto** 0);

Sal : **out** std_logic);

End Divisor;

Architecture Estructural **of** Divisor **is**

Signal Q : std_logic_vector(3 **downto** 0);

Signal Mux : std_logic_vector(1 **downto** 0);

-- Se declaran los componentes

Component Contador **generic** (N : natural := 8);

port (Clk, Clear : **in** std_logic;

Q : **out** std_logic_vector(N-1 **downto** 0));

End Component;

Component Mux2a1 **port** (

S : **in** std_logic;

E : **in** std_logic_vector(1 **downto** 0);

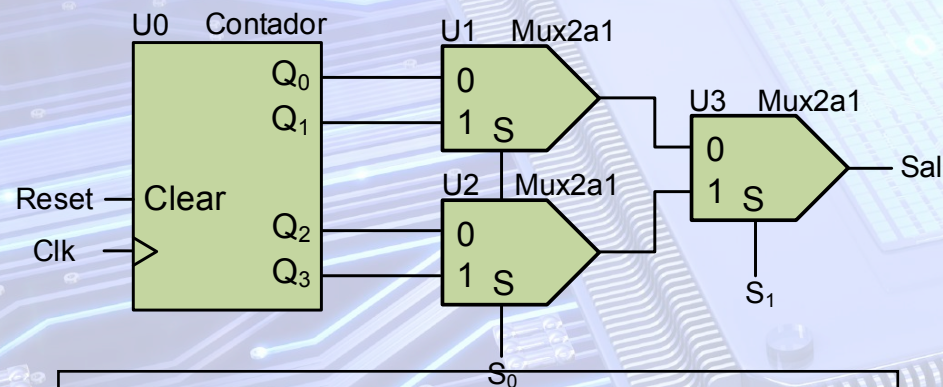
Y : **out** std_logic);

End Component;

5. Descripción de sistemas jerárquicos mediante múltiples ficheros. Divisor de frecuencia

Fichero fuente Divisor.vhd (continuación)

 La etiqueta es equivalente a la referencia del componente en el esquema eléctrico.



 La etiqueta permite instanciar varias veces un mismo componente. En el ejemplo, 3 Mux2a1.

Begin

-- se instancia el contador U0

U0 : Contador **generic map** (N => 4)

port map (Clk => Clk,
Clear => Reset,
Q => Q);

-- se instancia el mux U1

U1 : Mux2a1 **port map** (
E => Q(1 **downto** 0),
S => S(0),
Y => Mux(0));

-- se instancia el mux U2

U2 : Mux2a1 **port map** (
E => Q(3 **downto** 2),
S => S(0),
Y => Mux(1));

-- se instancia el mux U3

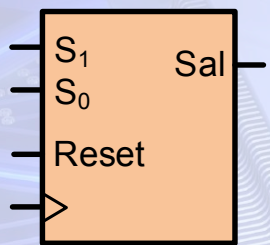
U3 : Mux2a1 **port map** (
E => Mux,
S => S(1),
Y => Sal);

End Estructural;

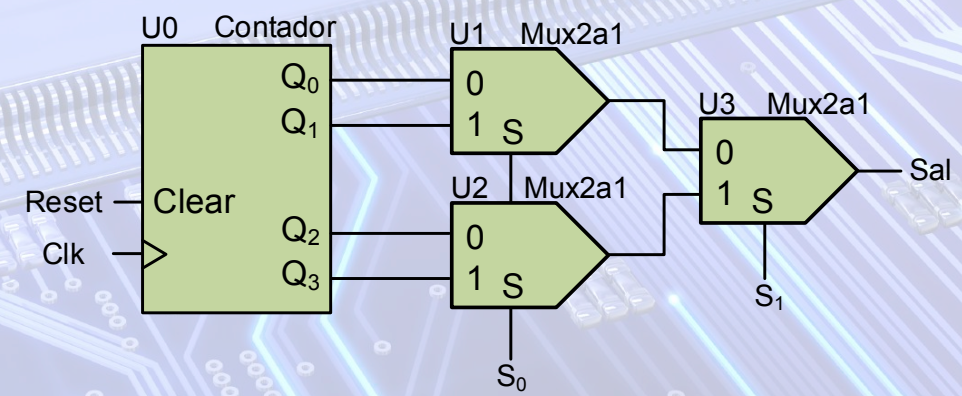
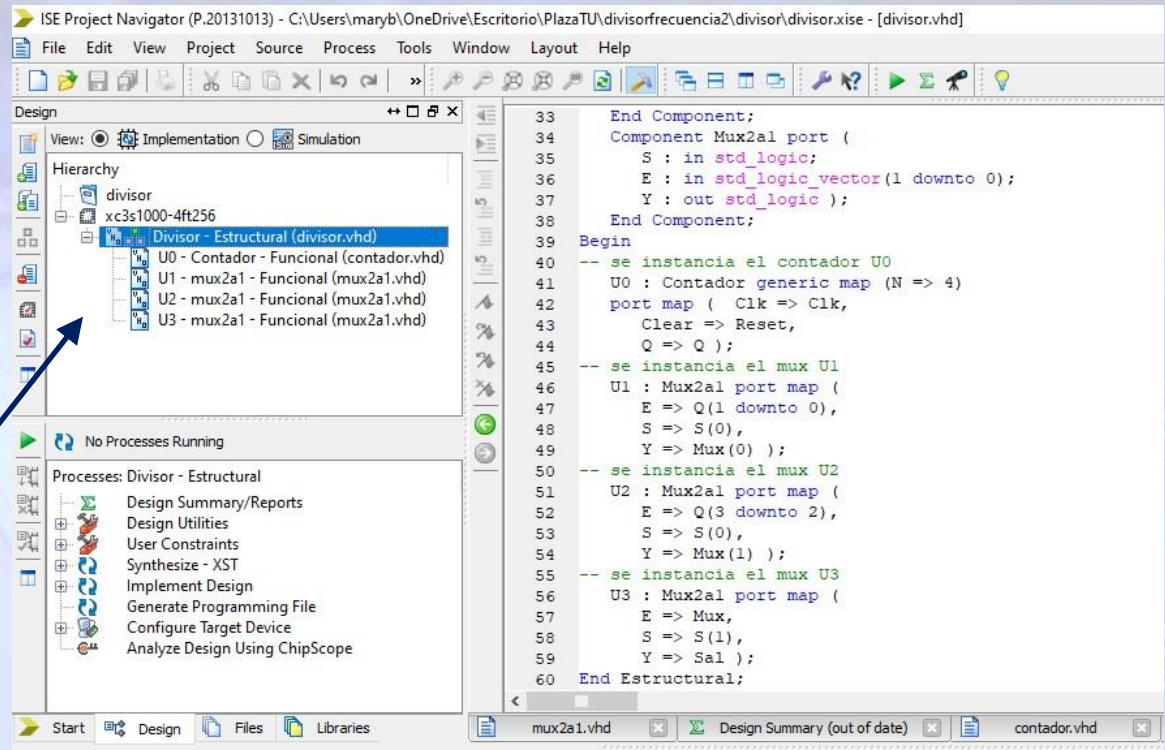
5. Descripción de sistemas jerárquicos mediante múltiples ficheros. Divisor de frecuencia

❖ Ventana del navegador de proyectos de ISE del Divisor

Divisor frecuencia



Muestra la jerarquía del diseño en forma de árbol



6. Packages (Paquetes). Descripción general

❖ Definición

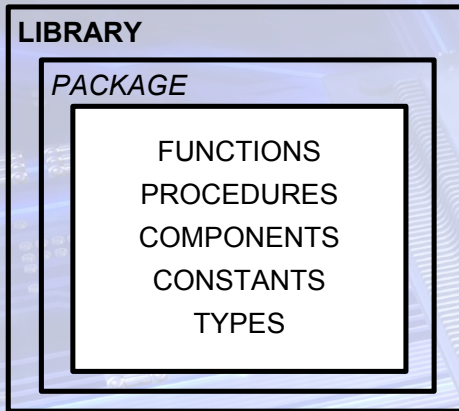
- Son áreas de almacenamiento que contiene una serie de declaraciones.

❖ Lugar de declaración de los packages:

- En el **mismo fichero fuente** que las restantes unidades de diseño.
- En un **fichero fuente diferente**.
 - ✓ Es el método más común y recomendable.
 - Facilita el mantenimiento y la claridad de los ficheros
 - ✓ Se compila en una librería.
 - Lo habitual es usar la **librería work**.

❖ Elementos que se incluyen en un package:

- Declaraciones de **tipo** y **subtipo**.
- Declaraciones de **constantes**.
- Declaraciones de **ficheros** (file) y **alias** (declara otro nombre alternativo para un objeto).
- Declaraciones de **componentes**.
- Declaraciones de **atributos**.
- **Funciones** (funtions) y **procedimientos** (procedures).



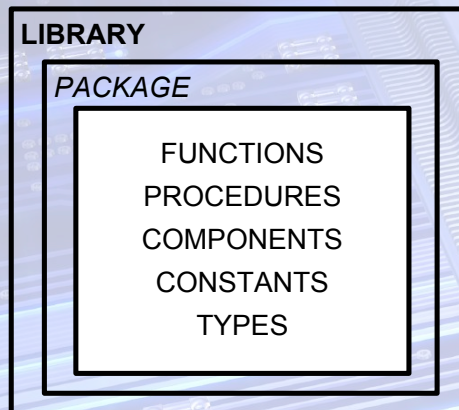
6. Packages (Paquetes). Descripción general

❖ Componentes de un package

- Consta de dos partes:
 - ✓ Declaración
 - ✓ Body (cuerpo)

❖ Declaración

- Contiene los **elementos que se exportan**: declaraciones de functions o procedures, **componentes**, **constantes** y **tipos**.
- Es **semejante a la entity**
- **Sintaxis**

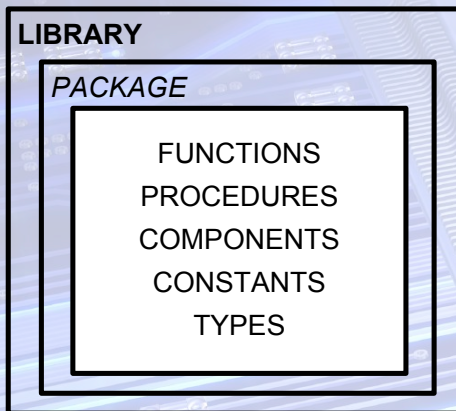


```
package nom_package is  
    declaraciones  
end nom_package;
```

6. Packages (Paquetes). Descripción general

✦ Body

- Forma parte de un package en el que se declara una **function** o un **procedure**.
- Define el **comportamiento** de la function o procedure.
- Es **equivalente a la architecture**.
- **Sintaxis**

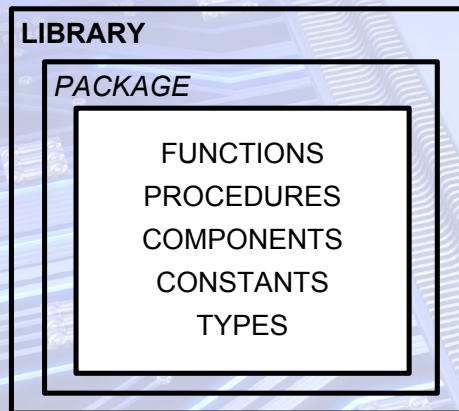


```
package body nom_package is
    declaraciones
    Descripción (body) de la función
    Descripción (body) del procedimiento
end nom_package;
```

- **Visibilidad de los elementos de un package**
 - Para usarlos en otras unidades de diseño debe incluirse una cláusula **use** antes de cada unidad de diseño.

6. Packages (Paquetes). Descripción general

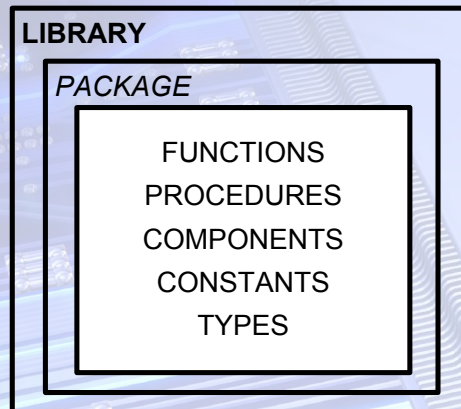
❖ Ejemplo. Package compuesto solamente por la parte de declaración



```
package declara_rom is
  Type palabra is array (7 downto 0) of std_logic;
  Type array_rom is array (0 to 7) of palabra;
  Constant rom : array_rom :=
    ( X"00",
      X"03",
      X"45",
      X"AD",
      X"87",
      X"44",
      X"B9",
      X"66" );
end declara_rom;
```

6. Packages (Paquetes). Descripción general

❖ Ejemplo. Package compuesto por la declaración y el body



```

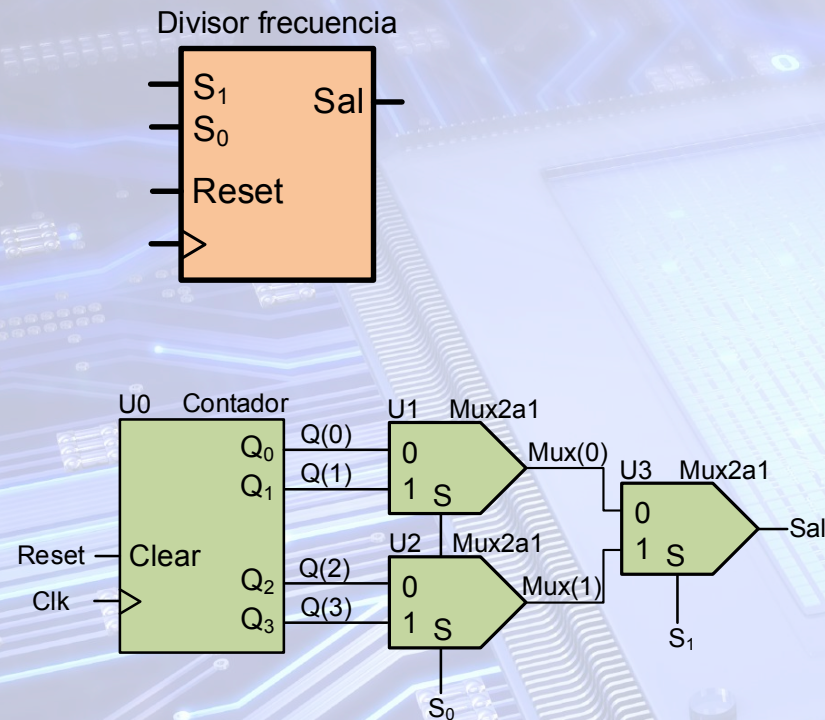
Library ieee;
Use ieee.std_logic_1164.all;

package reg8 is
    subtype byte is std_logic_vector(0 to 7);
    constant CLEAR8 : byte := (others => '0');
    procedure dff8 (signal Rst, Clk : in std_logic;
        signal D : in byte;
        signal Q : out byte);
end reg8;

package body reg8 is
    procedure dff8 (signal Rst, Clk : in std_logic;
        signal D : in byte;
        signal Q : out byte) is
    begin
        if Rst = '1' then
            Q <= CLEAR8;
        elsif Clk = '1' and Clk'event then
            Q <= D;
        end if;
    end dff;
end reg8;
  
```

6. Packages (Paquetes). Descripción de sistemas jerárquicos

- ❖ Se usará un package para declarar los componentes
 - Se crea el fichero vhd **decla_comp.vhd**



```

-- Package decla_comp declara los componentes
-- del nivel inferior
Library ieee;
Use ieee.std_logic_1164.all;
Package decla_comp is

-- Se declaran los componentes

Component Contador generic (N : natural := 8);
  port (
    Clk, Clear : in std_logic;
    Q : out std_logic_vector(N-1 downto 0) );
End Component;

Component Mux2a1 port (
  S : in std_logic;
  E : in std_logic_vector(1 downto 0);
  Y : out std_logic );
End Component;

End decla_comp;
  
```

6. Packages (Paquetes). Descripción de sistemas jerárquicos

❖ Antes de poder instanciarse los componentes en la arquitectura del nivel superior (fichero Divisor.vhd) debe hacerse visible el package.

- El package se ha compilado en la **librería work**.
- Se pone una **cláusula use** antes de la entidad del nivel superior.
- **Sintaxis de la cláusula use**

```
Use nombre_librería.nombre_package.nombre_componente;
```

- Cláusulas **use** de nuestro diseño

```
Use work.decla_comp.Contador;
```

```
Use work.decla_comp.Mux2a1;
```

- Se puede usar **all** en la cláusula **use** en vez del nombre_comp

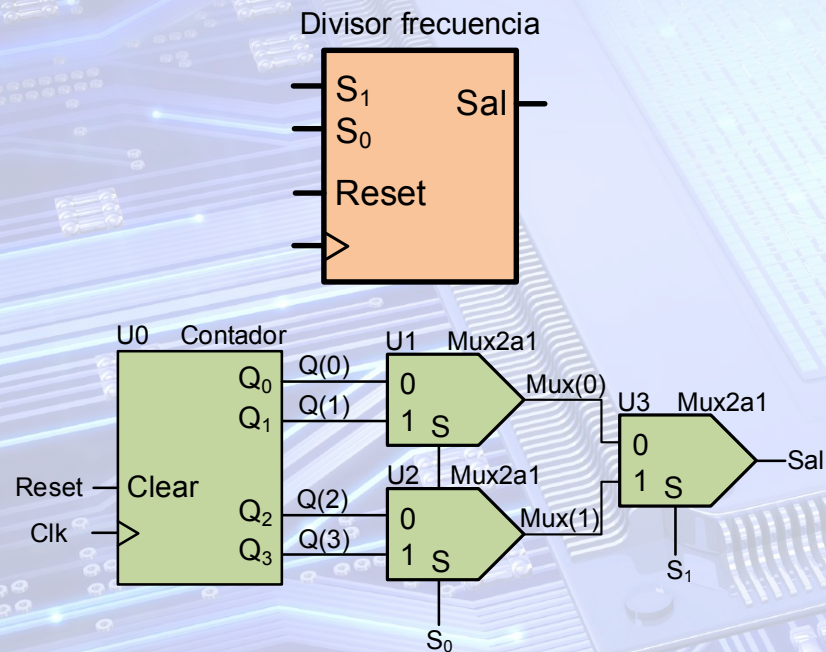
```
Use work.decla_comp.all;
```

- ✓ Hace visibles todos los componentes declarados en el package.
- ✓ Se usa una sola cláusula use.
- ✓ Es el método más común.

6. Packages (Paquetes). Descripción de sistemas jerárquicos

❖ Fichero fuente Divisor.vhd

- Hace visible la declaración de componentes en el package decla_comp.



```
-- Nivel raíz
Library ieee;
Use ieee.std_logic_1164.all;
Use work.decla_comp.all;

Entity Divisor is port (
    Reset, Clk : in std_logic;
    S : in std_logic_vector(1 downto 0);
    Sal : out std_logic );
End Divisor;

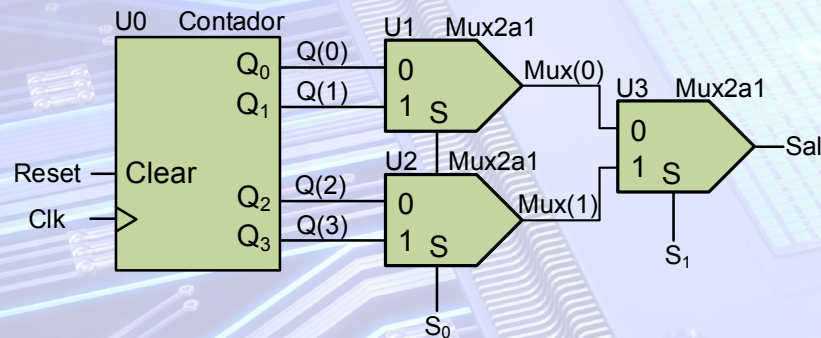
Architecture Estructural of Divisor is
    Signal Q : std_logic_vector(3 downto 0);
    Signal Mux : std_logic_vector(1 downto 0);
Begin
```

💡 No se declara los componentes, ya que se ha hecho en el package decla_comp.

6. Packages (Paquetes). Descripción de sistemas jerárquicos

❖ Fichero fuente
Divisor.vhd
(Continuación)

- La etiqueta es equivalente a la referencia del componente en el esquema eléctrico.



- La etiqueta permite instanciar varias veces un mismo componente. En el ejemplo, 3 Mux2a1.

-- se instancia el contador U0

```
U0 : Contador port map (
  Clk => Clk,
  Clear => Reset,
  Q => Q );
```

-- se instancia el mux U1

```
U1 : Mux2a1 port map (
  E => Q(1 downto 0),
  S => S(0),
  Y => Mux(0) );
```

-- se instancia el mux U2

```
U2 : Mux2a1 port map (
  E => Q(3 downto 2),
  S => S(0),
  Y => Mux(1) );
```

-- se instancia el mux U3

```
U3 : Mux2a1 port map (
  E => Mux,
  S => S(1),
  Y => Sal );
```

End Estructural;

7. Diseño parametrizable en VHDL

- Para mejorar la productividad en el diseño de los sistemas complejos se puede usar la **técnica de reuso de diseños**.
 - ✓ Muchos diseños tienen **partes comunes**, aunque realicen funciones que a priori pueden ser diferentes.
 - ✓ Se puede diseñar y verificar estas partes una vez, almacenarlas en una librería y después reusarlas en otros diseños o aplicaciones.
 - Estas **partes comunes se declaran como componentes** y después se conectan entre sí y con otros para crear un nuevo diseño.
 - Mediante los **generics se pueden parametrizar estos componentes** y adaptarlos a las especificaciones del diseño.
 - En el caso de un contador se puede declarar un generic para definir el número de bits, e incluso la dirección de la cuenta.
- VHDL tiene dos statements, **for generate** y **for loop**, que pueden usarse para describir sistemas o componentes con estructuras iterativas.

7. Statement Generate. Introducción

- Un statement generate es un statement concurrente que permite repetir o generar statements concurrentes, que describen componentes básicos, para obtener un sistema u otro componente más complejo.
- Hay dos tipos de statements generate:
 - ✓ **For generate**
 - Crea un circuito repitiendo un elemento hardware básico.
 - ✓ **If generate**
 - Especifica si se crea o no un elemento hardware opcional.
- Muchos circuitos se pueden implementar como un conjunto repetitivo de bloques básicos.
 - ✓ **Generalmente muestran una estructura regular:**
 - Cadena de conexión en cascada de una dimensión.
 - Conexión tipo árbol.
 - Matriz de dos dimensiones.
 - ✓ **Se puede expandir fácilmente la estructura aumentando el número de iteraciones.**
 - En esto se basa el concepto de diseño parametrizable.
 - El statement for generate se usa para este tipo de circuitos.

7. Statement Generate. For Generate

❖ Utilidad.

Describir sistemas o componentes en los que se repiten statements concurrentes.

❖ Sintaxis.

Etiqueta_generate:

```
for índice_bucle in rango_bucle generate
```

```
    [declaraciones
```

```
    begin ]
```

```
    statements_concurrentes;
```

```
end generate [Etiqueta_generate];
```

- Repite los statements concurrentes del bucle un número fijo de veces.
- La etiqueta es obligatoria.
- Rango_bucle especifica el rango de valores entre los límites izquierdo y derecho.
 - ✓ El rango es estático. Se debe especificar en el momento de la síntesis.
 - ✓ El rango generalmente depende de generic o constantes previamente declaradas.

7. Statement Generate. For Generate

❖ Sintaxis y características

- La variable **índice_bucle** se declara automáticamente.
 - ✓ Indica la iteración actual.
 - ✓ Se actualiza automáticamente con cada iteración del bucle.
 - ✓ Toma sucesivamente los valores de **rango_bucle** en cada una de las iteraciones empezando desde el valor más a la izquierda.
- **Los statements concurrentes pueden ser de cualquier tipo.**
 - ✓ Statements de asignación de señal.
 - ✓ Instanciación de componentes.
 - ✓ Procesos
 - ✓ Otros statements generate.
- **Los statements concurrentes describen una parte del circuito iterativo**

Etiqueta_generate:

```
for índice_bucle in rango_bucle generate  
    [declaraciones  
begin ]  
    statements_concurrentes;  
end generate [Etiqueta_generate];
```

💡 **Es semejante a for loop.**

💡 En éste, sólo se puede repetir statements secuenciales y de asignación de valores a señales y/o a variables dentro de un process.

7. Statement Generate. For Generate

❖ Sintaxis y características

- La variable **índice_bucle** se declara automáticamente.
 - ✓ Indica la iteración actual.
 - ✓ Se actualiza automáticamente con cada iteración del bucle.
 - ✓ Toma sucesivamente los valores de **rango_bucle** en cada una de las iteraciones empezando desde el valor más a la izquierda.
- Los **statements concurrentes** pueden ser de cualquier tipo.
 - ✓ Statements de asignación de señal.
 - ✓ Instanciación de componentes.
 - ✓ Procesos
 - ✓ Otros statements generate.
- Los **statements concurrentes** describen una parte del circuito iterativo

Etiqueta_generate:

```
for índice_bucle in rango_bucle generate  
    [declaraciones  
begin ]  
    statements_concurrentes;  
end generate [Etiqueta_generate];
```

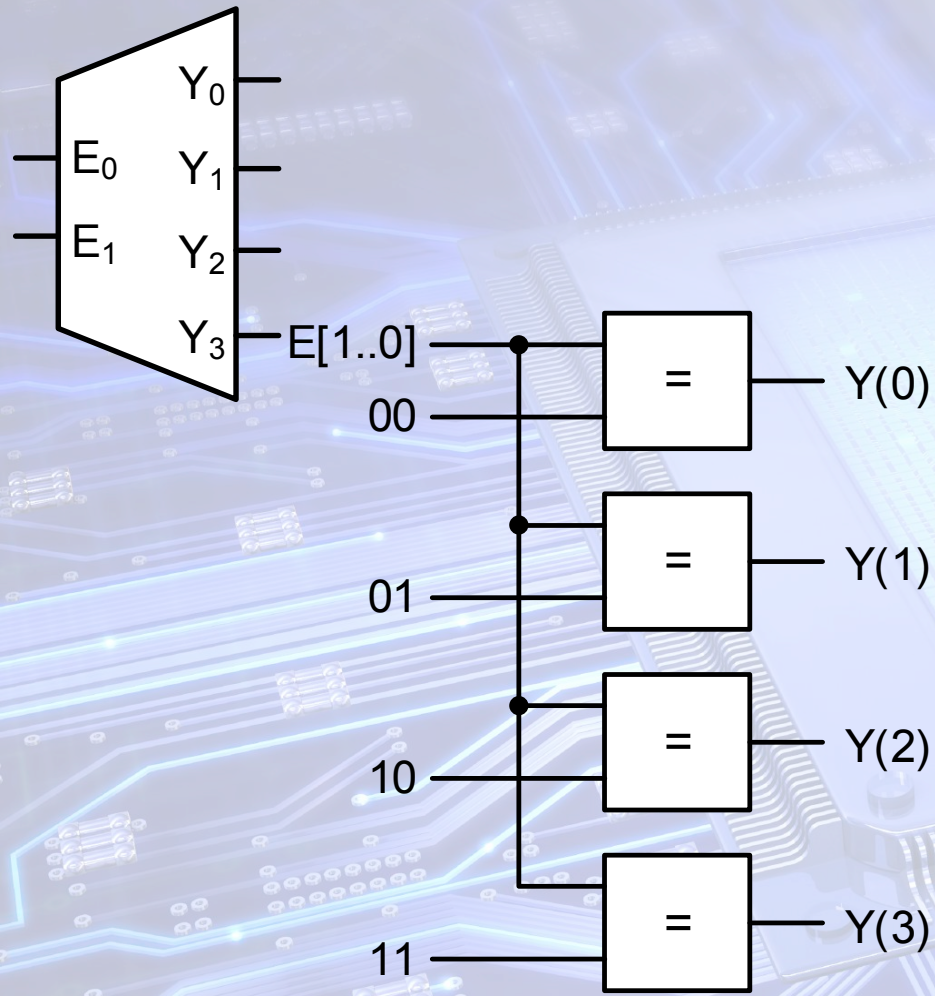
💡 Es semejante a for loop.

💡 En éste, sólo se puede repetir statements secuenciales y de asignación de valores a señales y/o a variables dentro de un process.

7. Statement Generate. For Generate

❖ Ejemplos

Decodificador binario de n a 2^n



- Partimos de un decod de 2 a $2^2 = 2$ a 4
 - ✓ Una salida (Y_i) se activa si el valor de entrada (E) coincide con el de su subíndice (i).
 - ✓ Se usará un comparador para cada salida.
- **Statement concurrente a repetir.**
 - ✓ Usando el package **Numeric_Std**

$$Y(i) \leq '1' \text{ when } i = \text{to_integer}(\text{unsigned}(E)) \text{ else '0'};$$
 - ✓ Usando el package **Std_logic_unsigned**

$$Y(i) \leq '1' \text{ when } i = \text{Conv_integer}(E) \text{ else '0'};$$

7. Statement Generate. For Generate. Ejemplos

❖ Decodificador binario de n a 2^n

■ Código parametrizado

Library ieee;

Use ieee.std_logic_1164.all;

Use ieee.numeric_std.all;

Entity Decod_bin is

generic(N : natural := 2);

port (

E : in std_logic_vector(N-1 downto 0);

Y : out std_logic_vector(2**N-1 downto 0));

End Decod_bin;

Architecture Gen_arquitectura of Decod_bin is

Begin

Gen_decod:

for i in 0 to 2**N-1 generate

Y(i) <= '1' when i = to_integer(unsigned(E))
else '0';

end generate;

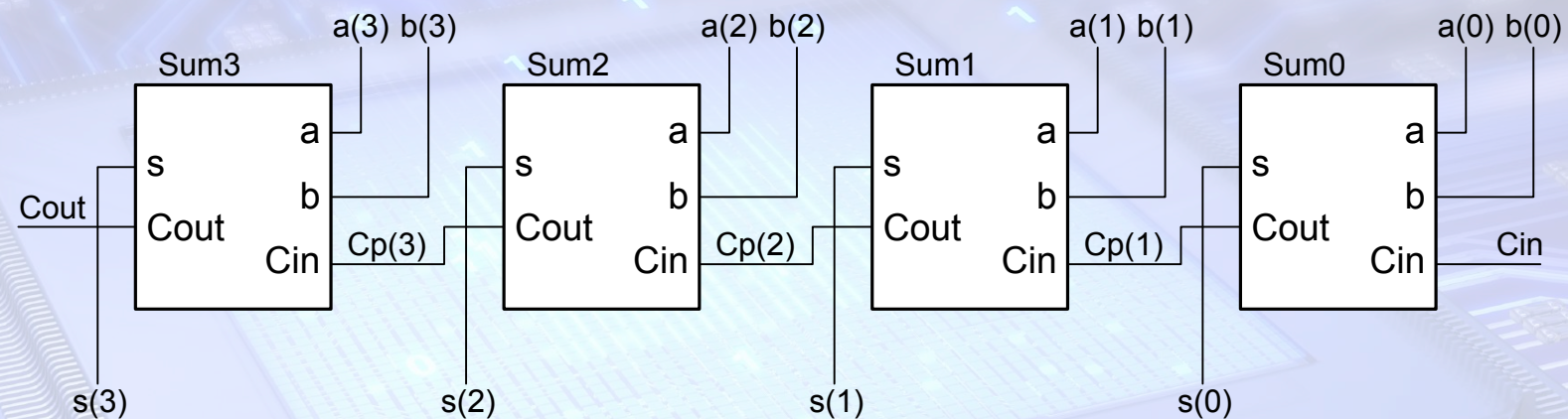
End Gen_arquitectura;

■ Mediante el **generic N** parametrizamos el tamaño del decodificador.

- El **generic N** indica el número de iteraciones del bucle.
- Se describe la función lógica de cada una de las salidas.

7. Statement Generate. For Generate. Ejemplos

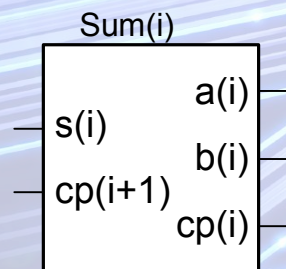
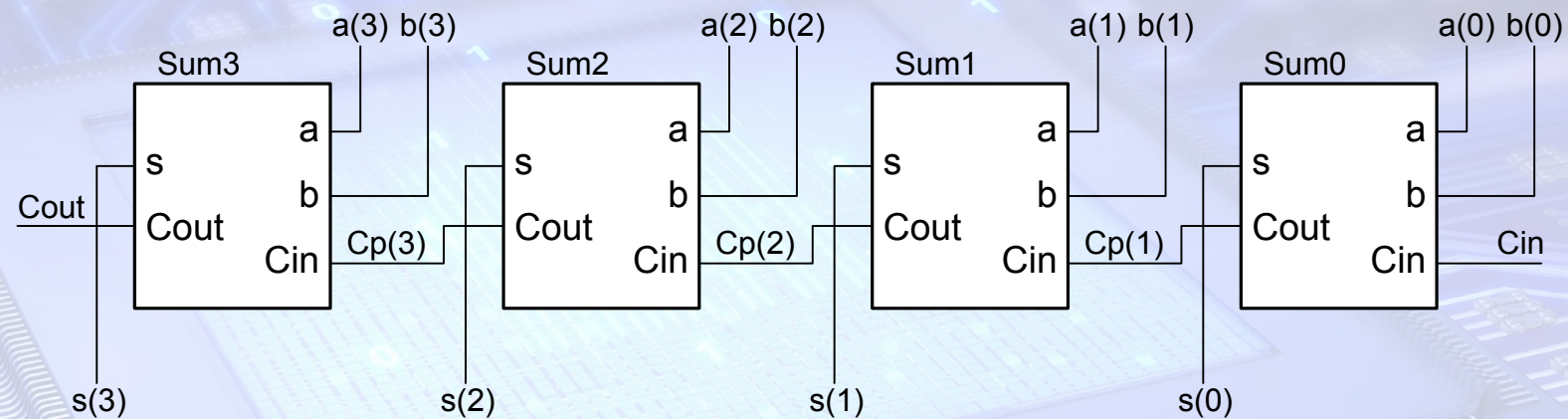
- ❖ Sumador binario con propagación serie del acarreo
 - Se parte de un sumador de 4 bits



- Bloque básico:
 - ✓ Sumador completo de 1 bit.
- Patrón de etapas. Dos opciones:
 - ✓ Descripción del funcionamiento.
 - A partir de las funciones lógicas de las salidas del sumador completo.
 - ✓ Descripción de su estructura.
 - Como un sistema jerárquico a partir del componente sumador completo.

7. Statement Generate. For Generate. Ejemplos

- ❖ Sumador binario con propagación serie del acarreo
 - Descripción del funcionamiento. Estilo Data flow



✓ Statements concurrentes a repetir.

- Funciones lógicas de las salidas

$S = a \text{ xor } b \text{ xor } \text{Cin}$

$\text{Cout} = (a \text{ and } b) \text{ or } (a \text{ and } \text{Cin}) \text{ or } (b \text{ and } \text{Cin})$

- Para expresarlo de forma iterativa:

- Se debe declarar una **señal del tipo array (Cp)** de 5 bits para conectar la salida de acarreo de una etapa con la entrada de acarreo de la siguiente.

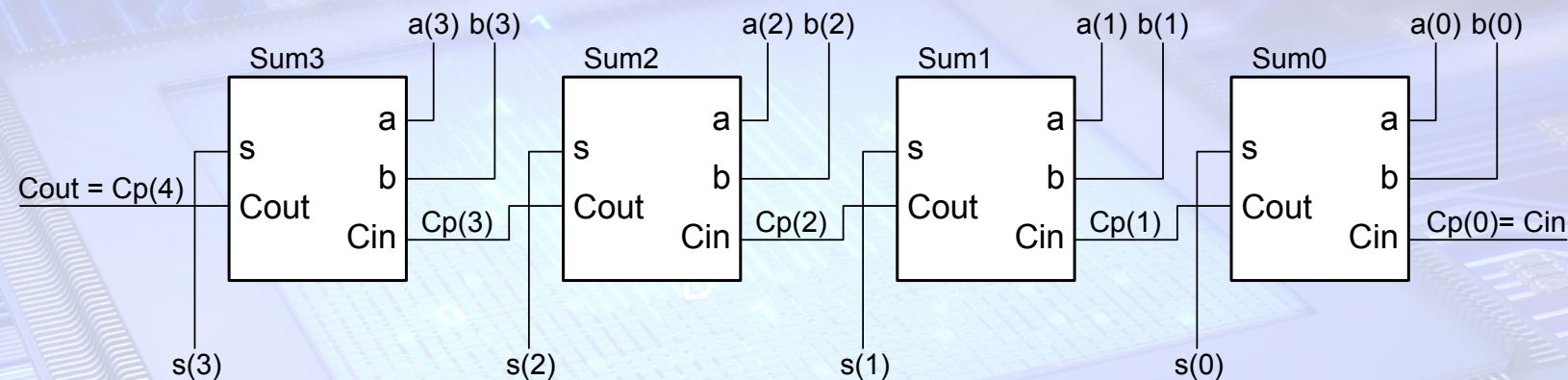
signal Cp : std_logic_vector(4 downto 0);

- Cin de Sum0 se asigna a Cp(0) y Cp(4) a Cout.

7. Statement Generate. For Generate. Ejemplos

- ❖ Sumador binario con propagación serie del acarreo
 - Descripción del funcionamiento. Estilo Data Flow

✓ Diagrama lógico final a describir



✓ Statements concurrentes a repetir.

- Statements de asignación a las señales de salida.

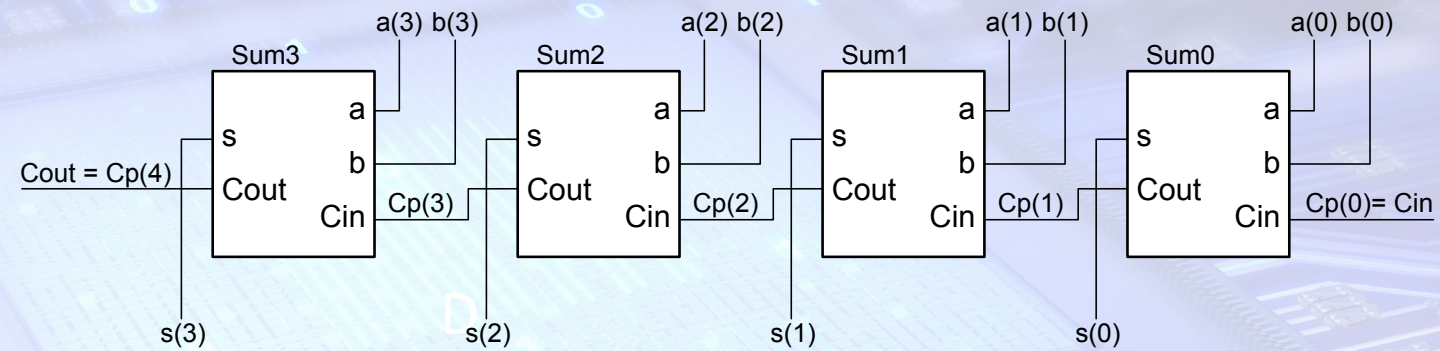
$$S(i) \leq a(i) \text{ xor } b(i) \text{ xor } Cp(i);$$

$$Cp(i+1) \leq (a(i) \text{ and } b(i)) \text{ or } (a(i) \text{ and } Cp(i)) \text{ or } (b(i) \text{ and } Cp(i));$$

7. Statement Generate. For Generate. Ejemplos

- ❖ Sumador binario con propagación serie del acarreo
 - Descripción del funcionamiento. Estilo Data Flow
 - Sumador genérico de n bits

- La entidad es la misma para ambas descripciones.
- Mediante el **generic N** se describe un sumador genérico de n bits.



```

Library ieee;
Use ieee.std_logic_1164.all;
Entity Suma_bin is
  generic(N : natural);
  port (
    a : in std_logic_vector(N-1 downto 0);
    b : in std_logic_vector(N-1 downto 0);
    Cin : in std_logic;
    Cout : out std_logic;
    s : out std_logic_vector(N-1 downto 0) );
End Suma_bin;
  
```

```

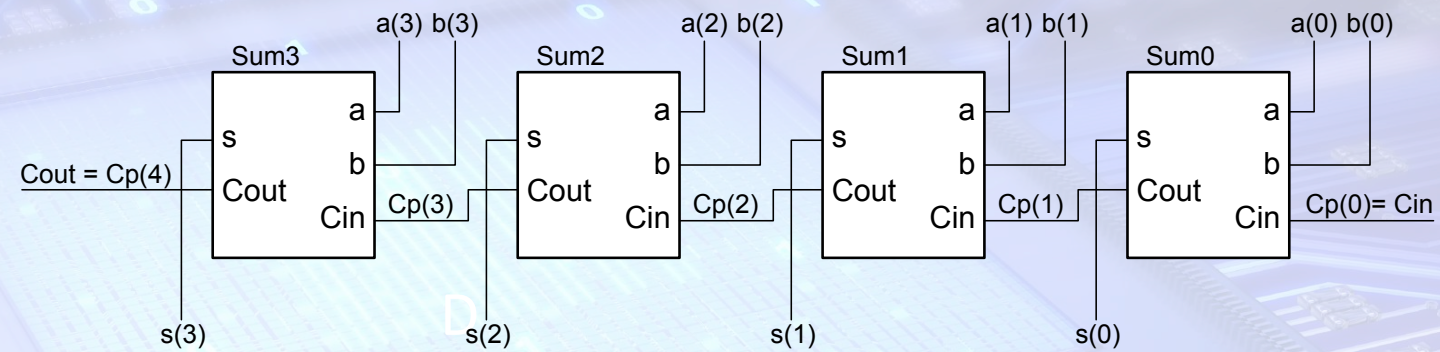
Architecture Gen_arquitectura of Suma_bin is
  signal Cp : std_logic_vector(N downto 0);
Begin
  Cp(0) <= Cin;
  Gen_adder: for i in 0 to N-1 generate
    S(i) <= a(i) xor b(i) xor Cp(i);
    Cp(i+1) <= (a(i) and b(i)) or
               (a(i) and Cp(i)) or
               (b(i) and Cp(i));
  end generate;
  Cout <= Cp(N);
End Gen_arquitectura;
  
```

7. Statement Generate. For Generate. Ejemplos

❖ Sumador binario con propagación serie del acarreo

▪ Descripción de la estructura. Estilo jerárquico

▪ Diagrama lógico final a describir



- Primero se describe en el estilo dataflow el sumador completo.

```

Library ieee;
Use ieee.std_logic_1164.all;
Entity Suma_Com is
  port (
    a : in std_logic;
    b : in std_logic;
    Cin : in std_logic;
    Cout : out std_logic;
    s : out std_logic );
End Suma_Com;
  
```

Architecture Funcional of Suma_Com is
Begin

```

    S <= a xor b xor Cin;
    Cout <= (a and b) or
             (a and Cin) or
             (b and Cin);
  
```

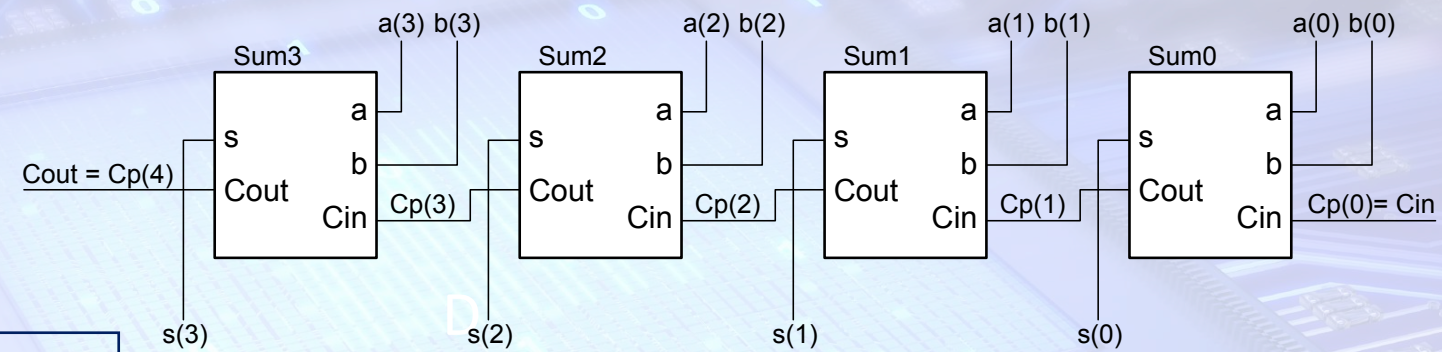
End Funcional;

7. Statement Generate. For Generate. Ejemplos

❖ Sumador binario con propagación serie del acarreo

■ Descripción de la estructura. Estilo jerárquico

■ Sumador genérico de n bits



- En el nivel superior se instancian 4 copias del componente y se conectan.
- La entidad es igual que la anterior.

Architecture Gen_arquitectura of Suma_bin is

Component Suma_Com is port (

a : in std_logic;

b : in std_logic;

Cin : in std_logic;

Cout : out std_logic;

s : out std_logic);

signal Cp : std_logic_vector(N downto 0);

Begin

-- Continuación de la arquitectura del nivel superior

Cp(0) <= Cin;

Gen_adder: for i in 0 to N-1 generate

Suma: Suma_Com port map (

a => a(i),

b => b(i),

Cin => Cp(i),

s => s(i),

Cout => Cp(i+1));

end generate;

Cout <= Cp(N);

End Gen_arquitectura;

8. Statement Generate. If Generate

❖ Utilidad

Especificar si se incluye o no, uno o varios statements concurrentes en la implementación final de un sistema o componente, que se puede describir de forma iterativa mediante statements concurrentes.

❖ Sintaxis

Etiqueta_generate:

```
if Boolean_exp generate
```

```
  [declaraciones
```

```
begin ]
```

```
  statements_concurrentes
```

```
end generate [Etiqueta_generate] ;
```

- La etiqueta es obligatoria.
- **Boolean_exp** es una expresión que devuelve un valor de tipo boolean.
 - ✓ Si es **true**, los statements concurrentes internos se tienen en cuenta en la síntesis, por lo que los circuitos correspondientes se incluyen en la implementación final.
 - ✓ Si es **false**, se excluyen los statements de la síntesis y, por tanto, los circuitos correspondientes en la implementación final.

8. Statement Generate. If Generate

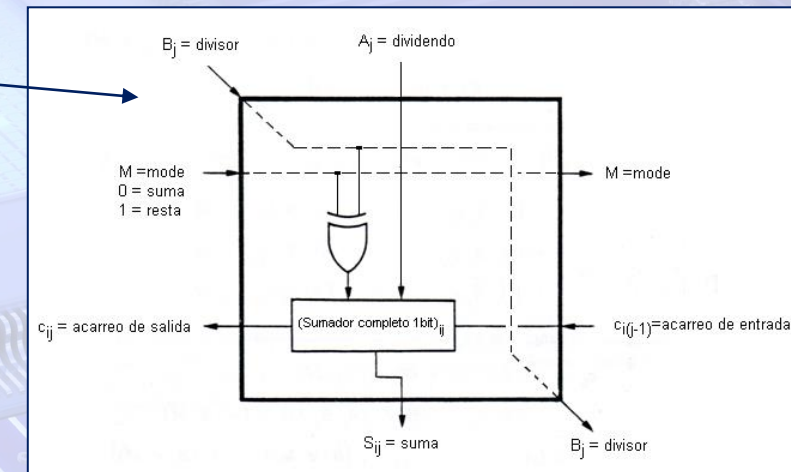
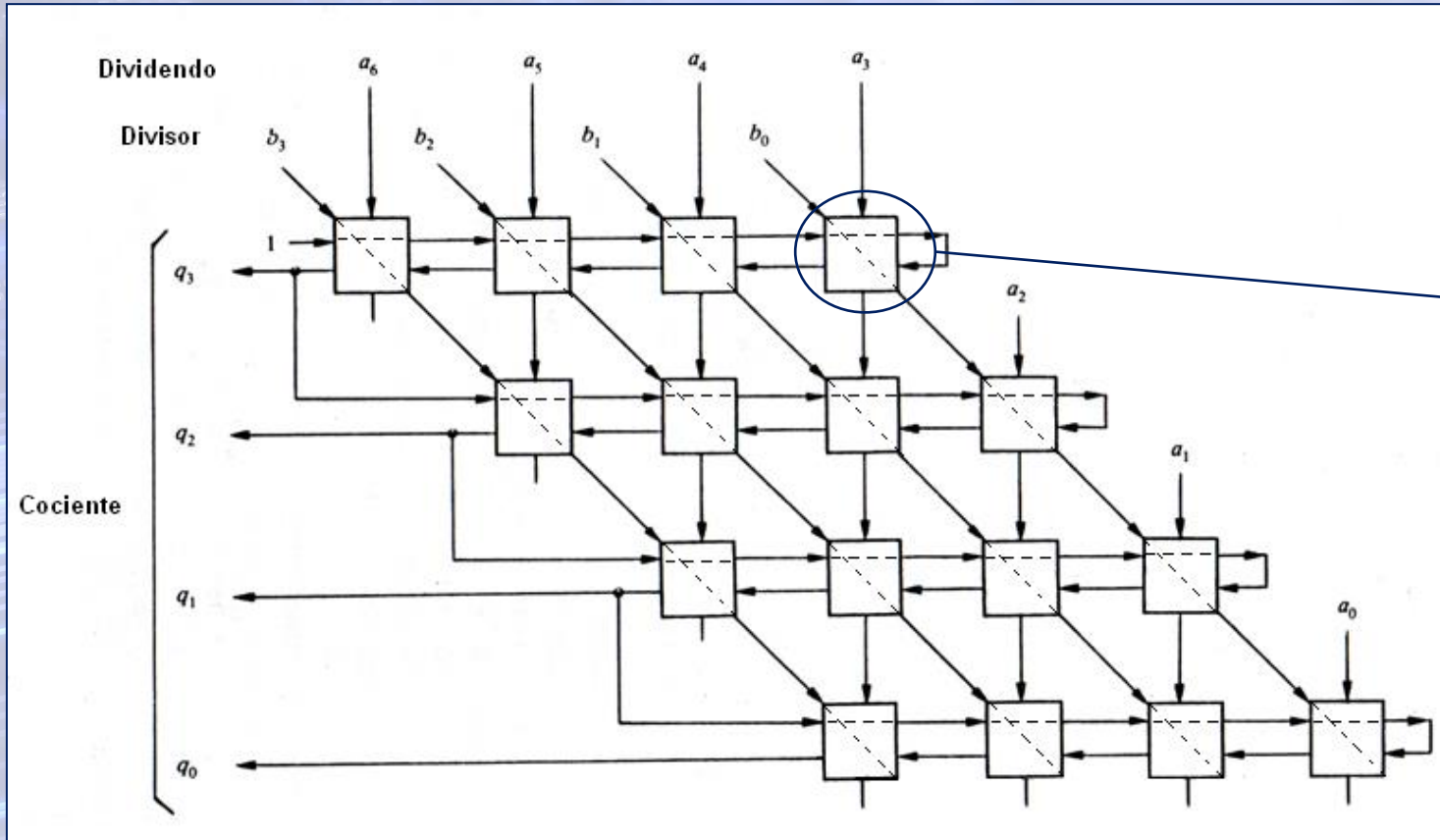
❖ Sintaxis

```
Etiqueta_generate:  
    if Boolean_exp generate  
        [declaraciones  
        begin ]  
        statements_concurrentes  
    end generate [Etiqueta_generate] ;
```

- No hay una cláusula else para permitir otra opción cuando **Boolean_exp** devuelve un valor false.
 - ✓ Para incluir una de dos posibles opciones de statements concurrentes, se debe usar dos statements if generate separados.
- **Boolean_exp** debe ser una expresión estática,
 - ✓ La herramienta de síntesis debe saber si los correspondientes statements concurrentes se incluyen o no en implementación física.
 - ✓ La expresión generalmente se describe en función de algún o algunos generics.

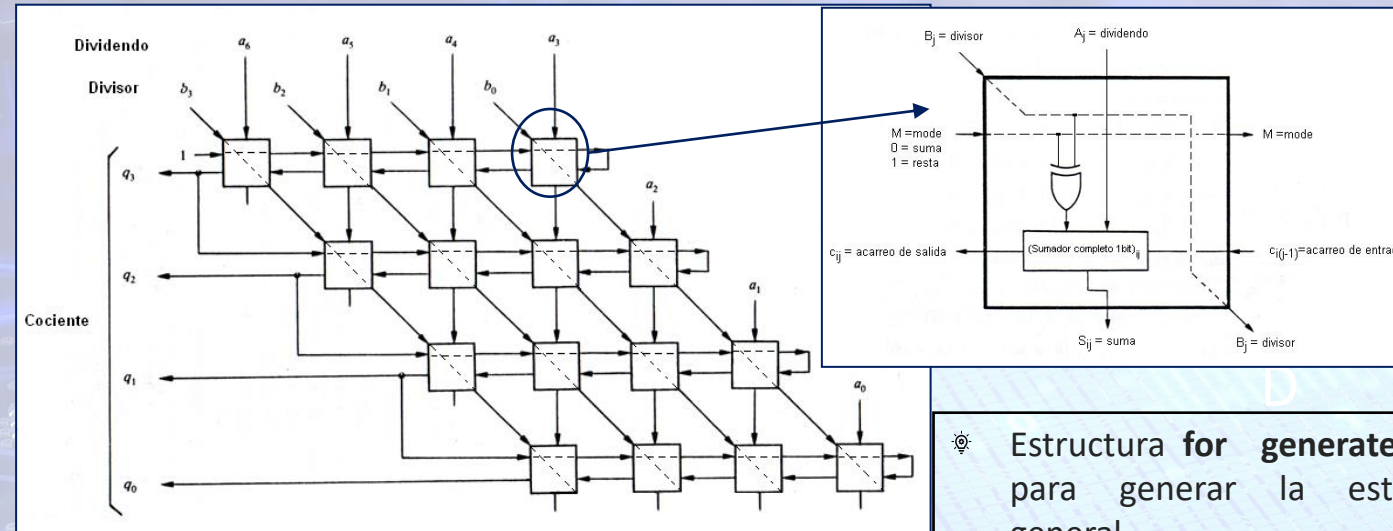
8. Statement Generate. If Generate. Ejemplos

- ❖ **División binaria.** Algoritmo sin restauración para el desarrollo de la división de dos números binarios (7 bits para el dividendo y 4 bits para el divisor) basado en la realización de desplazamientos y operaciones de sumas y restas



8. Statement Generate. If Generate. Ejemplos

❖ División binaria



entity celda is

Port (a, b, c, M: in std_logic;
S, cout: out std_logic);

end celda;

architecture Vision_funcional of celda is

begin

S <= (a xor b) xor (c xor M);

cout <= (a and c) or ((b xor M) and (c or a));

end Vision_funcional;

⚡ Estructura for generate usada para generar la estructura general

⚡ Estructura if generate usada para especificar excepciones a la regla de conexión

type matriz is array (0 to 3, 0 to 3) of std_logic;

signal M : std_logic_vector(3 downto 0);

signal a, b, c: matriz;

M(3) <= '1';

G1: for i in 0 to 3 generate

G2: if (i=3) generate

M(i) <= '1';

G3: for j in 0 to 3 generate

G4: if (j=0) generate

cell: celda port map (Dividendo(j+i), Divisor(j), M(i), M(i), s(i,j), c(i,j));

end generate;

G5: if (j>0) generate

cell: celda port map (Dividendo(j+i), Divisor(j), c(i,j-1), M(i), s(i,j), c(i,j));

end generate;

end generate;

end generate;

G6: if (i<3) generate

M(i) <= c(i+1,3);

G7: for j in 0 to 3 generate

G8: if (j=0) generate

cell: celda port map (Dividendo(i), Divisor(j), M(i), M(i), s(i,j), c(i,j));

end generate;

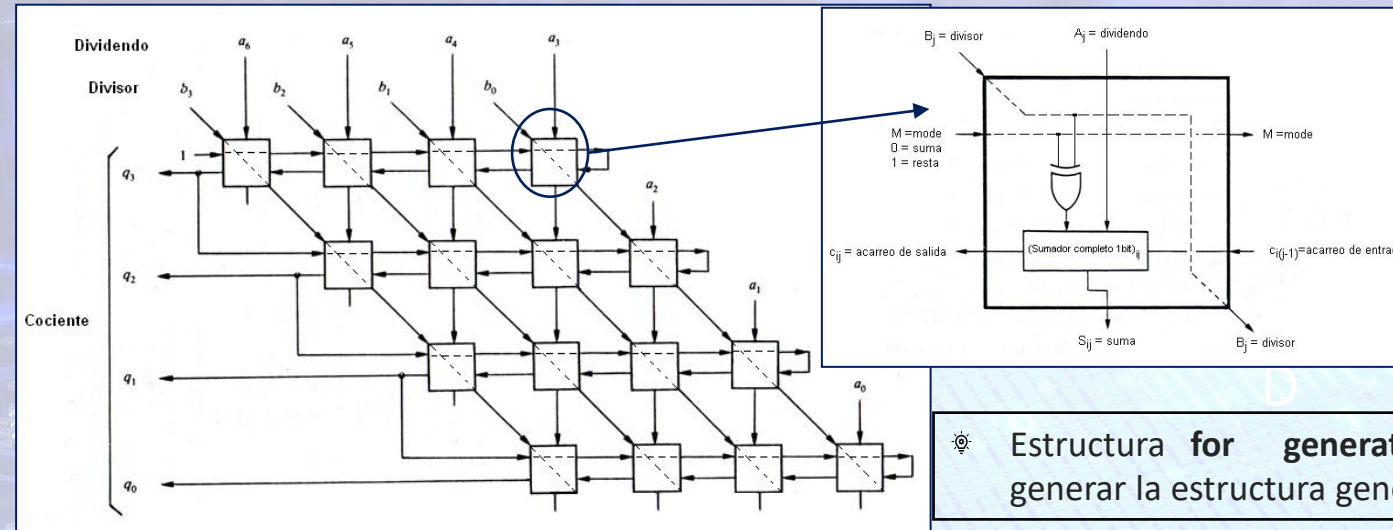
G9: if (j>0) generate

cell: celda port map (s(i+1,j-1), Divisor(j), c(i,j-1), M(i), s(i,j), c(i,j));

.....

8. Statement Generate. If Generate. Ejemplos

- ❖ **División binaria.** Algoritmo sin restauración para el desarrollo de la división de dos números binarios (7 bits para el dividendo y 4 bits para el divisor) basado en la realización de desplazamientos y operaciones de sumas y restas



type matriz is **array** (0 to 3, 0 to 3) of **std_logic**;

signal M : **std_logic_vector**(3 **downto** 0);

signal a, b, c: **matriz**;

M(3)<='1';

lab1: **for** i in 0 to 3 **generate**

lab2: **if** (i<3) **generate**

lab3: **for** j in 0 to 3 **generate**

lab4: **if** (j>0) **generate**

a(i,j)<=suma_resta(a(i+1,j-1), b(i+1,j), c(i,j-1), M(i));

c(i,j)<=acarreo(a(i+1,j-1), b(i+1,j), c(i,j-1), M(i));

b(i,j)<=b(i+1,j);

end generate;

lab5: **if** (j=0) **generate**

a(i,j)<=suma_resta(Dividendo(i), b(i+1,j), M(i), M(i));

c(i,j)<=acarreo(Dividendo(i), b(i+1,j), M(i), M(i));

b(i,j)<=b(i+1,j);

end generate;

end generate;

end generate;

lab6: **if** (i=3) **generate**

lab7: **for** j in 0 to 3 **generate**

lab8: **if** (j>0) **generate**

a(i,j)<=suma_resta(Dividendo(j+i), Divisor(j), c(i,j-1), M(i));

...

⚡ Estructura **for generate** usada para generar la estructura general

⚡ Estructura **if generate** usada para especificar excepciones a la regla de conexión

function suma_resta(A,B,Cin,M: **STD_LOGIC**) **return** **STD_LOGIC** **is**
begin

return((A XOR B) XOR (M XOR Cin));

end suma_resta;

function acarreo(A,B,Cin,M: **STD_LOGIC**) **return** **STD_LOGIC** **is**
begin

return ((A AND Cin) OR ((M XOR B) AND (A OR Cin)));

end acarreo;